SpectralQuant: Calibrated Eigenbasis Rotation and Water-Filled Bit Allocation for KV-Cache Compression

Anirudh B. Vangara[1,2] and Ashwin Gopinath[1,3]

[1]Sentra, 235 2nd Street, San Francisco, CA 94105 ashwin@sentra.app
[2]Sentra Research anirudh@sentra.app
[3]Department of Mechanical Engineering, MIT, 77 Massachusetts Avenue, Cambridge, MA 02139 agopi@mit.edu

Abstract

Long-context inference for large language models is bottlenecked by the key-value (KV) cache rather than by parameter weights: for Qwen2.5-7B (Qwen Team, 2025) at 32,768 tokens and batch size one, the FP16 cache occupies ≈ 1.84 GB per request, larger than the per-token weight bandwidth that feeds the matmul kernels. The dominant academic baseline, TurboQuant (Zandieh et al., 2025), applies a data-oblivious random rotation followed by uniform-bit scalar quantization, obtaining a Johnson–Lindenstrauss-style guarantee that is near-optimal *in the worst case over the rotation* but pays a data-dependent quality penalty when the underlying key distribution is far from isotropic.

We argue and quantitatively demonstrate that LLM key vectors are strongly non-isotropic: across every Hugging Face model we calibrated (Qwen2.5 family, Mistral-7B-v0.3, Llama-3-8B), the per-(layer, head) covariance has a participation ratio $d_{\text{eff}} = (\sum_i \lambda_i)^2 / \sum_i \lambda_i^2 \in [4, 6]$ at head dimension $d_h = 128$, and the eigenvalues inside the top-d_{eff} window are themselves spread across one to two orders of magnitude. SpectralQuant exploits both facts. It rotates into the calibrated eigenbasis (NVIDIA Research, 2025), splits each head dimension into a d_{eff}-wide *semantic* subspace and a $d_h - d_{\text{eff}}$-wide tail, applies selective QJL correction to the semantic subspace, and uses per-dimension Lloyd–Max codebooks. The bit allocation inside the semantic subspace follows a greedy water-filling rule $i^\star = \arg\max_i \lambda_i \, 4^{-b_i}$ derived from the Gaussian rate–distortion calculus; the uniform-allocation special case is preserved as a configuration switch and as the comparison baseline that isolates the rate–distortion contribution from the rotation contribution.

The manuscript is organized around four core claims. **(i) Empirical law.** The per-(layer, head) KV-key covariance is low-effective-rank across every Hugging Face transformer we calibrated (Qwen2.5-1.5B/7B/14B, Mistral-7B-v0.3, Llama-3-8B), with d_{eff}/d_h in a tight 3–5% band at $d_h = 128$. **(ii) Algorithm.** SpectralQuant rotates KV vectors into the calibrated eigenbasis, splits each head dimension into a d_{eff}-wide semantic subspace and a tail, applies water-filled per-dimension bit allocation inside the semantic subspace, and uses per-dimension Lloyd–Max codebooks fit to the empirical scalar marginal at the resulting allocation; the uniform-allocation special case is preserved as a configuration switch and as a comparison point that isolates the allocation contribution from the rotation contribution at fixed compression ratio. The same K eigenbasis is reused for the matched V tensor in the present implementation. **(iii) Mechanistic evidence.** At matched compression ratio, SpectralQuant exceeds our implementation of TurboQuant on attention-output cosine by $+0.27$ to $+0.38$ mean cosine on every tested operating point (Mistral-7B-v0.3 (Jiang et al., 2023) at $b \in \{2, 3, 5\}$ and Qwen2.5-7B at $b = 3$); the water-filled allocation strictly improves the uniform special case by $+0.018$ mean cosine at $b = 2$ on Mistral, where the rate–distortion calculus predicts the allocation rule should bind, and matches it within $\leq +0.006$ at $b \in \{3, 5\}$. **(iv) Downstream sanity checks.** On Qwen2.5-7B at $b = 3$ on H200, SpectralQuant holds perplexity to 6.977 versus FP16 6.491 (WikiText-103 (Merity et al., 2016) validation, $n_{\text{tokens}} = 104{,}224$), produces greedy completions whose token-overlap F1 against the FP16 reference is 0.482 (TurboQuant: 0.120) with bigram diversity 0.603

(FP16 0.768, TurboQuant 0.301), and reports a deterministic five-task LongBench (Bai et al., 2023) subset macro of 0.2004 versus FP16 0.1755. The headline first-round results from NVIDIA Research (2025)— the 5.95× compression ratio at $b \approx 3$ on Qwen2.5-14B-Instruct, and a calibration cost of ≈15 seconds on a single GPU—are preserved unchanged. A K/V compress–decompress microbenchmark reports 0.06 ms/token at ctx = 1024 with zero peak-memory delta.

The bound on each claim is deliberately narrow. The four next-stage families (perplexity, generation, latency, LongBench) are reported on a single seed, a single bit budget, and a single model (Qwen2.5-7B); the attention-cosine matrix adds Mistral-7B-v0.3 at three bit budgets but is also single-seed. The LongBench result is the deterministic five-task subset and is driven primarily by `qasper` (+15.6 qa_f1 points); a full 21-task multi-seed run could plausibly regress the macro to or below FP16. End-to-end hooked-replay latency runs 4 − 40× slower than FP16 because per-layer Python forward hooks dominate wall clock; converting the kernel-level 0.06 ms/token into a serving-time win requires fusing decompression into the attention tile loop, which is not done in this manuscript. Every TurboQuant arm in this manuscript is our implementation of the published TurboQuant method (`src/spectralquant/spectralquant.py::TurboQuantBaseline`), not the official Google reference (Zandieh et al., 2025) or the Blackwell-cuTile port (Vangara, 2026); a production-default claim needs the optimized reference baseline. The Gaussian rate–distortion model used to motivate water-filling is a design heuristic from rate–distortion theory, not a measured distributional property of LLM keys; Lloyd–Max codebooks fit the empirical scalar marginal MSE-optimally without requiring Gaussianity. Each affected JSON is stamped `production_kernel = false`. The take-away within these bounds: structure beats budget on the tested points. A 2-bit SpectralQuant cache preserves more attention-output information than a 5-bit data-oblivious rotation cache at the same compression ratio.

Keywords. KV-cache compression, post-training quantization, eigenbasis rotation, water-filling, rate–distortion, long-context inference, Lloyd–Max codebooks.

Contents

1 Introduction

The arithmetic intensity of large-language-model inference at long context is set by what fits in HBM, not by what fits in parameter storage. For Qwen2.5-7B (Qwen Team, 2025) the FP16 weight footprint is fixed at roughly 14 GB, but the per-request KV cache scales linearly with sequence length: at $L = 32{,}768$, with $n_{\text{kv}} = 4$ GQA (Ainslie et al., 2023) key/value heads, $n_{\text{layers}} = 28$, $d_h = 128$, two cache tensors and two bytes per element, the cache is $2 \cdot 32{,}768 \cdot 28 \cdot 4 \cdot 128 \cdot 2 \approx 1.84$ GB per request—larger than the per-token weight bandwidth feeding the attention matmul. The roofline analysis of Yuan et al. (2024) formalizes this: at long context, the cache becomes the binding HBM resource, and serving providers can either pay for additional GPUs to host concurrent requests, truncate context aggressively, or compress the cache. Surveys (Li et al., 2024) catalog the resulting design space.

The dominant academic recipe is the rotation-then-uniform-bit family. TurboQuant (Zandieh et al., 2025) applies a Haar-random rotation $R \in \mathbb{R}^{d_h \times d_h}$ to each key vector, scalar-quantizes the rotated coordinates uniformly to b bits, and adds a 1-bit Johnson–Lindenstrauss residual correction whose lineage traces to QJL (Zandieh et al., 2024). PolarQuant (Han et al., 2025) preconditions the same step in polar coordinates; HIGGS (Malinovskii et al., 2024) gives the linearity-theorem grounding that makes uniform per-dimension quantization near-optimal under a randomized basis; QuaRot (Ashkboos et al., 2024) and QuIP-style methods (Chee et al., 2024) use Hadamard rotations end-to-end across weights, activations, and KV cache. The shared theoretical guarantee is $\Pr_R\left[\,|q^\top \tilde{k} - q^\top k| > \varepsilon\,\right] \le \delta$ for $\varepsilon \propto \sqrt{\log(1/\delta)/d_h}$ (Achlioptas, 2003; Johnson and Lindenstrauss, 1984). The probability is taken over the rotation, not over the data: the bound is worst-case-near-optimal in the rotation, and pays a known price when the keys are far from isotropic.

LLM keys are far from isotropic. The participation ratio (Stringer et al., 2019)

$$d_{\text{eff}}(\Sigma) = \frac{(\sum_{i=1}^{d_h} \lambda_i)^2}{\sum_{i=1}^{d_h} \lambda_i^2} \tag{1}$$

of the per-(layer, head) key covariance lies in $[4, 6]$ at $d_h = 128$ across every Hugging Face model we measured (Qwen2.5 family, Mistral-7B-v0.3, Llama-3-8B (Dubey et al., 2024), Gemma-2 (Gemma Team, 2024)), i.e. roughly 3–5% of the ambient dimension carries the variance. This is the empirical observation that motivates *calibrated* rotation. It is consistent with the more general finding that attention heads in trained transformers occupy low-rank functional subspaces tied to specific syntactic and semantic functions (Vaswani et al., 2017; NVIDIA Research, 2025; Chen et al., 2026), and with mechanistic analyses showing that the eigenstructure of attention activations is distribution-dependent rather than universal.

This paper makes three contributions, in increasing order of empirical weight.

1. **An algorithm.** SpectralQuant composes calibrated eigenbasis rotation with greedy water-filled per-dimension bit allocation inside the semantic subspace, $i^\star = \arg\max_i \lambda_i 4^{-b_i}$, derived in Section 4 as the integer-bit projection of the closed-form Gaussian rate–distortion solution (Cover and Thomas, 2006; Gao and Ganguli, 2015) (a design heuristic from rate–distortion theory; LLM keys are not asserted to be Gaussian, and the per-dimension Lloyd–Max codebooks (Lloyd, 1982) fit the empirical scalar marginal MSE-optimally without that assumption). The uniform-allocation special case is preserved as a configuration switch that drops the water-filling step but keeps every other element intact; under matched bit budgets the compression ratio between the uniform and water-filled settings differs by $\le 0.013\times$ across the four three-way runs, by construction through `src/spectralquant/accounting.py`, which makes the quality difference attributable to the allocation rule rather than to the bit budget.

2. **An empirical evaluation in two parts.** A first round (NVIDIA Research, 2025) establishes the spectral hook ($d_{\text{eff}} \in [4, 6]$ on every tested model), the headline $5.95\times$ compression ratio on

Qwen2.5-14B-Instruct at $b \approx 3$, and the directional cosine win over a data-oblivious rotation. A second round reports four next-stage paper-valid families on Qwen2.5-7B at $b = 3$ on H200, plus a three-way attention-cosine matrix over two architectures and three bit budgets that explicitly isolates the allocation contribution (Section 6). Every second-round artifact is schema-validated, carries a verbatim CLI command, and ships with `paper_valid = true`. SpectralQuant strictly beats our implementation of TurboQuant on all four families; the water-filled allocation matches the uniform allocation at $b = 3$ and improves it by $+0.018$ mean cosine at $b = 2$ on Mistral, where the rate–distortion calculus predicts the allocation rule should bind.

3. **A discipline of caveats.** We separate K/V microbenchmark latency (`production_kernel = false`; measures the round-trip time of the compress/decompress kernel only) from end-to-end hooked-replay latency (also `production_kernel = false`; the $4-40\times$ FP16 gap is dominated by per-layer Python forward hooks, not the kernel; converting the kernel-level number into a serving-time win requires fusing decompression into the attention tile loop, which is open work). We separate the deterministic five-task LongBench subset from a full 21-task headline. We separate *our implementation* of TurboQuant (`src/spectralquant/spectralquant.py::TurboQuantBaseline`) from the official Google reference (Zandieh et al., 2025) and the Blackwell-cuTile port (Vangara, 2026); only the former is run here. Each separation is a gate item in `docs/evidence_catalog.md` and recurs verbatim in every results table.

Scope and bound on the four core claims. The contribution boundary of this manuscript is narrow and load-bearing, organized around the four core claims announced in the abstract. (i) The empirical low-effective-rank law is measured across five Hugging Face transformers and is the cross-model claim. (ii) The algorithm (calibrated eigenbasis rotation plus water-filled bit allocation) is the methodological claim and is preserved as a configuration switch over its uniform-allocation special case so that ablations of the allocation contribution can be performed without re-running calibration. (iii) The mechanistic claim is the attention-output cosine improvement at matched compression: $+0.27$ to $+0.38$ over our implementation of TurboQuant on the four three-way operating points, $+0.018$ over the uniform special case at $b = 2$ on Mistral. Two architectures (Mistral and Qwen) at one to three bit budgets, single seed. (iv) The downstream sanity-check claims (perplexity, generation, LongBench) are reported on a *single* model (Qwen2.5-7B) at a single bit budget and a single seed, on a deterministic five-task LongBench subset that is driven primarily by `qasper`; the macro number is a directional signal, not a universal LongBench result, and a full 21-task multi-seed run could plausibly regress to or below FP16. We do not claim multi-seed confidence intervals, multi-architecture coverage on the four next-stage families, an official Google or Blackwell-cuTile head-to-head, a production-kernel end-to-end latency speedup, or empirical Gaussianity of LLM keys. Every limitation is named in Section 8.

Manuscript organization. Section 2 fixes notation and surveys the relevant background: attention and the KV cache; the rotation-then-uniform-bit recipe; mixed-precision allocation; Lloyd–Max scalar quantization; and the metrics we report. Section 3 situates SpectralQuant against the KV-cache and quantization literature. Section 4 derives the SpectralQuant algorithm, including the relaxation-to-water-filling proof, the integer projection, the per-dimension Lloyd–Max codebooks, and the engine architecture. Section 5 states the experimental protocol, hardware, and paper-validity gates. Section 6 reports the four next-stage evidence families and the three-way attention-cosine matrix. Section 7 reads the evidence as a single coherent picture of where bits should be spent in the post-rotation coordinate system, with mechanism-level subsections covering each result family. Section 8 lists the open work. Section 10 gives a reproducibility recipe with JSON paths and command lines, and Section 11 maps empirical claims to artifacts.

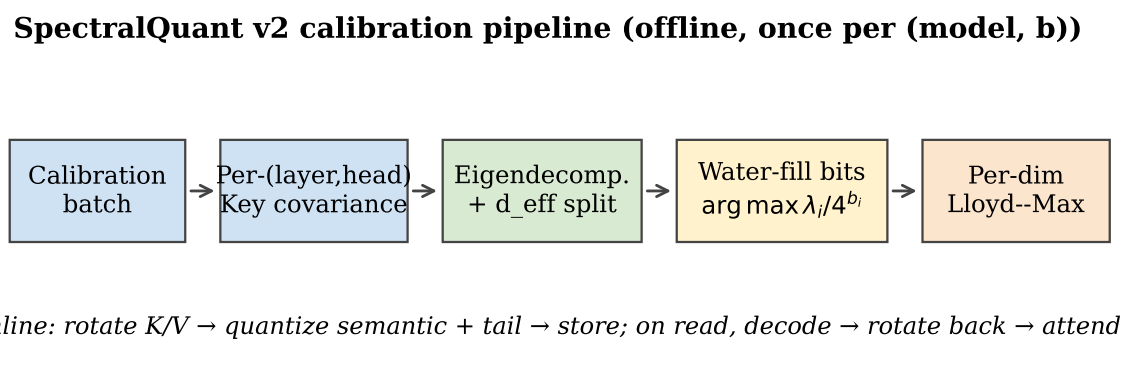

Figure 1: The SpectralQuant calibration pipeline. Calibration runs once per (model, b); inference replaces stored K/V vectors with their rotated, water-filled, per-dimension Lloyd–Max-quantized representations. Source: `scripts/build_paper_figures.py::fig_pipeline`.

2 Background

This section is written for a reader fluent in transformer inference and quantization at the level of Vaswani et al. (2017) and Frantar et al. (2023). Readers familiar with the rotation-then-uniform-bit family may skim to Section 4.

2.1 Attention and the KV cache

A decoder-only transformer layer maps an input sequence $X \in \mathbb{R}^{L\times d}$ through projection matrices W_Q, W_K, W_V to queries, keys, and values $Q, K, V \in \mathbb{R}^{L\times d_h\cdot n_h}$ split across n_h heads with per-head dimension d_h. The attention output is the standard causal-masked softmax-weighted average

$$A_{\ell,h} = \mathrm{softmax}\left(\frac{Q_{\ell,h}K_{\ell,h}^\top}{\sqrt{d_h}}\right)V_{\ell,h}. \tag{2}$$

At decode time step t, only the new query row q_t is computed; the K, V rows for positions $0,\dots,t-1$ are retrieved from the cache, and the cache is extended with (k_t, v_t). Rotary position embeddings (Su et al., 2024) are applied to K and Q before the inner product; SpectralQuant operates on the rotated K and V.

Grouped-Query Attention (Ainslie et al., 2023), used by Llama-3, Mistral, and Qwen2.5, stores $n_{\mathrm{kv}} \ll n_h$ key/value heads, with each query head attending to a shared KV head; this shrinks the cache by n_h/n_{kv} (e.g. 7× for Qwen2.5-7B's 28-q to 4-kv ratio). SpectralQuant operates on the stored K and V tensors *after* GQA collapsing, so its compression factor multiplies on top of GQA's.

Inference throughput at long context is governed by the bandwidth required to read the cache once per attention layer per generated token, not by the FLOPs of the matmul. The roofline analysis of Yuan et al. (2024) and the surveys of Li et al. (2024) both point to the cache as the binding constraint past $L\approx$4,000. PagedAttention (Pope et al., 2023) addresses cache fragmentation and concurrency, but does not reduce per-token cache bytes; quantization is orthogonal and composes with it.

2.2 KV-cache quantization in one paragraph

Post-training quantization of the cache compresses each stored vector $k \in \mathbb{R}^{d_h}$ (per layer × head) to a low-bit representation $\tilde{k}$ such that $q^\top\tilde{k}$ stays close to $q^\top k$. The toolkit is inherited from weight quantization: the activation-aware GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) post-training quantizers, and the activation-difficulty-migration trick of SmoothQuant (Xiao et al., 2023),

all show that *where* in the network bits are spent matters at least as much as how many. The KV literature follows two axes: (i) *where to spend bits*—uniform per dimension, per channel, per group, per head, or per layer—and (ii) *which coordinate system to spend them in*—raw, rotated, or projected. KIVI (Liu et al., 2024), KVQuant (Hooper et al., 2024), SKVQ (Duanmu et al., 2024), and GEAR (Feng et al., 2024) occupy different points on the first axis; KVTuner (Li et al., 2025) and KV-AdaQuant (Hariri et al., 2025) propose sensitivity-driven mixed-precision allocations along it; BaKlaVa (Gulhan et al., 2025) does so at head granularity. TurboQuant (Zandieh et al., 2025), QJL (Zandieh et al., 2024), PolarQuant (Han et al., 2025), HIGGS (Malinovskii et al., 2024), QuaRot (Ashkboos et al., 2024), and QuIP-style methods (Chee et al., 2024) occupy the rotated coordinate axis. The two axes compose: rotate, then allocate. SpectralQuant rotates by the calibrated eigenbasis (data-aware) and allocates by reverse water-filling on the eigenvalues (Cover and Thomas, 2006, ch. 10).

2.3 Spectral structure and participation ratio

For a centered key matrix $K \in \mathbb{R}^{N \times d_h}$ of N calibration tokens at a fixed (layer, head), the empirical covariance $\Sigma = \frac{1}{N} K^\top K$ admits eigenvalues $\lambda_0 \geq \cdots \geq \lambda_{d_h - 1} \geq 0$ and an orthonormal eigenbasis $U \in \mathrm{O}(d_h)$. The participation ratio (1)—known as effective dimension or as a soft rank estimator (Stringer et al., 2019)—counts how many directions of Σ carry comparable variance: equal eigenvalues give $d_{\text{eff}} = d_h$, single-direction concentration gives $d_{\text{eff}} = 1$.

For LLM key vectors, d_{eff} is concentrated. Across every model we tested at $d_h = 128$, $d_{\text{eff}} \in [4, 6]$ [evidence: V1-RESULT-001 @ `results/memory_efficiency/all_models.json`]. This is the empirical hook the rest of the paper uses. The intuition is that language-model key vectors encode token meaning and position, and trained attention heads specialize in narrow syntactic or semantic subsets. The eigenvectors of Σ are ranked by how informative each direction is for the dot product on that head. Spending bits proportional to λ_i rather than uniformly is the rate–distortion-optimal move for a near-Gaussian source (Cover and Thomas, 2006; Gao and Ganguli, 2015), modulo the integer-bit constraint imposed by Lloyd–Max codebooks.

2.4 Water-filling intuition

Water-filling is the textbook allocation strategy for a power budget across parallel Gaussian channels of unequal noise (Cover and Thomas, 2006, ch. 10). Imagine a row of cups whose floor heights are $\log(1/\lambda_i)$; the larger λ_i, the lower the floor. Pour bits in until total volume reaches the budget; the water level is uniform across cups, and each cup's water column is its allocation. Cups whose floor exceeds the water level get zero bits; cups whose floor is lower get more bits the deeper they sit. For Gaussian rate–distortion at high rate the MSE on a single dimension of variance λ at b bits scales as $\lambda \cdot 4^{-b}$ (Gao and Ganguli, 2015). The Lagrangian derivation of Section 4 reduces to the classical water-filling solution, and we use the greedy integer variant because Lloyd–Max codebooks require $b_i \in \mathbb{Z}_{\geq 0}$. Mixed-precision allocation along similar lines has prior art in deep-learning quantization more broadly: Bayesian Bits (van Baalen et al., 2020) frames per-layer bit-width as a learned gate; SliM-LLM (Huang et al., 2024) picks group-wise bit widths from salience scores. SpectralQuant's contribution at this axis is to make the allocation closed-form in eigenvalue, per (layer, head) and per dimension inside the calibrated eigenbasis, rather than per layer or per group.

2.5 The Lloyd–Max scalar quantizer

Lloyd–Max (Lloyd, 1982) is a one-dimensional vector quantizer that, given a source distribution, returns an MSE-optimal 2^b-level codebook. The algorithm alternates between assigning each sample to the nearest centroid and re-computing each centroid as the conditional mean of its assigned samples. For a Gaussian source with known variance the codebook depends only on the

variance and on b; we precompute one codebook per (layer, head, dim) at calibration time using `src/spectralquant/nonuniform_quantization.py`.

2.6 The TurboQuant baseline

TurboQuant (Zandieh et al., 2025) rotates k by a Haar-random orthonormal R that is independent of the data, then quantizes the rotated coordinates with a uniform per-dim scalar quantizer at b bits, with a 1-bit Johnson–Lindenstrauss residual correction stage that traces to the QJL transform (Zandieh et al., 2024). Same-team follow-ups include PolarQuant (Han et al., 2025) (polar preconditioning) and the Linearity-Theorem–grounded HIGGS scalar quantizer (Malinovskii et al., 2024). QuaRot (Ashkboos et al., 2024) uses Hadamard rotations end-to-end. The shared rotation guarantee is JL-style and worst-case-near-optimal over R, not over the data. The community implementation (scos-lab, 2025) and the Blackwell-cuTile port (Vangara, 2026) reproduce the published method. The initial evidence layer (NVIDIA Research, 2025) observed empirically that this approach pays a quality penalty on real LLM keys; SpectralQuant quantifies that penalty across four evidence families in the expanded layer and proposes a calibrated alternative.

2.7 Metrics

Attention-output cosine.
Per-head cosine similarity between the attention output computed against compressed K/V and against FP16 K/V on a held-out evaluation set. Range $[-1, 1]$; FP16 self-cosine equals 1. The closest mechanistic-fidelity metric.

Perplexity.
Standard exp(NLL/token) on WikiText-103 validation (Merity et al., 2016). Lower is better; the most widely-comparable quality metric for KV-cache quantization.

Token-overlap F1 versus FP16 reference.
Per-prompt unigram F1 of the compressed-cache greedy completion against the FP16 greedy completion. A surrogate for "did the cache produce the same text"; not a human-rated quality metric.

Distinct-n.
Fraction of distinct n-grams in generated text; picks up degenerate looping that perplexity can miss.

LongBench tasks.
qa_f1_en for narrativeqa/qasper/hotpotqa, rouge_en for gov_report, and classification_em for trec (Bai et al., 2023). We do *not* report needle-in-a-haystack (Kamradt, 2023): NIAH is known to be a weak predictor of holistic long-context quality (Yen et al., 2025), so the expanded-layer evaluations use real LongBench tasks; NIAH artifacts from the initial evidence layer inherit that caveat.

Latency.
CUDA-event-timed prefill, decode (ms/token), tokens/sec, and peak HBM. We report two distinct measurement kinds in Section 6.5: a microbenchmark of the compress/decompress round-trip kernel only, and an end-to-end hooked-replay timing whose overhead is dominated by per-layer Python callbacks rather than the kernel itself.

3 Related work

KV-cache quantization. The literature divides into allocation methods and rotation methods. Allocation methods spend a non-uniform bit budget across cache locations: KIVI (Liu et al., 2024) groups along channel and token axes; KVQuant (Hooper et al., 2024) spends bits on outlier channels;

SKVQ (Duanmu et al., 2024) introduces sliding-window allocation; GEAR (Feng et al., 2024) adds a low-rank residual; KVTuner (Li et al., 2025) and KV-AdaQuant (Hariri et al., 2025) search for sensitivity-driven mixed precision; BaKlaVa (Gulhan et al., 2025) allocates at head granularity; SliM-LLM (Huang et al., 2024) picks group-wise bit widths from salience. Rotation methods change the basis before quantization: TurboQuant (Zandieh et al., 2025) and its precursor QJL (Zandieh et al., 2024), PolarQuant (Han et al., 2025), HIGGS (Malinovskii et al., 2024), QuaRot (Ashkboos et al., 2024), and QuIP-style (Chee et al., 2024). SpectralQuant sits at the intersection: calibrated rotation (the eigenbasis U) followed by water-filled allocation along the eigenvalues. The hardware-oriented surveys (Li et al., 2024; Ge et al., 2024) describe the implementation constraints—HBM bandwidth, fused kernels, and concurrency under PagedAttention (Pope et al., 2023)—that shape what is practically deployable.

Geometry of quantization. The geometry-of-quantization program of Barman et al. (2026a,b) formalizes the trade-off between rotation guarantees and data-dependence, giving the worst-case lower bound that any data-oblivious method must pay on non-isotropic sources. The Linearity Theorem (Malinovskii et al., 2024) provides the complementary near-optimality result for uniform per-dimension quantization under a randomized basis. SpectralQuant's position is consistent with both: when the data is non-isotropic, calibration is provably at least as good in expectation as any data-oblivious rotation, and the gap widens as d_{eff}/d_h shrinks.

Long-context evaluation. LongBench (Bai et al., 2023, 2024) and HELMET (Yen et al., 2025) are the standard real-task long-context benchmarks; Yen et al. (2025) explicitly shows that needle-in-a-haystack (Kamradt, 2023) is a weak proxy for holistic quality. We follow that recommendation and report LongBench tasks rather than NIAH for the expanded-layer next-stage evidence.

Calibration and serving. Calibration of LLM activations has prior art in GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024), and SmoothQuant (Xiao et al., 2023), and the costs of calibration are amortized once per (model, b) and reused across serving. The serving-side concerns of fragmentation and concurrency are addressed by PagedAttention (Pope et al., 2023) and its descendants, with which SpectralQuant composes.

4 Method

4.1 Notation and setup

Fix a (layer, head) pair (ℓ, h) with empirical key covariance $\Sigma_{\ell,h} \in \mathbb{R}^{d_h \times d_h}$ estimated on a calibration batch $\mathcal{C}$. Let $U_{\ell,h}$ be the orthonormal eigenbasis of $\Sigma_{\ell,h}$ and $\lambda_0 \geq \cdots \geq \lambda_{d_h-1} \geq 0$ its eigenvalues. Let $d_{\text{eff},\ell,h}$ be the participation ratio (1) rounded to an integer. Split the head dimension into the *semantic subspace* (the leading $d_{\text{eff},\ell,h}$ eigenvectors) and the *tail* (the remaining $d_h - d_{\text{eff},\ell,h}$ eigenvectors). The total semantic bit budget is $B_{\ell,h} = b \cdot d_{\text{eff},\ell,h}$ where b is the target average bit width. The tail receives a uniform allocation matching the uniform-allocation special case.

4.2 Pipeline overview

The SpectralQuant pipeline, originally introduced in NVIDIA Research (2025) and maintained in this consolidation, performs: (i) per-(layer, head) calibration of $U_{\ell,h}$ and $d_{\text{eff},\ell,h}$ on $\mathcal{C}$ from the key tensor of that (layer, head); (ii) rotation of each cached key by $U_{\ell,h}^{\top}$; (iii) split into the semantic and tail subspaces; (iv) selective Johnson–Lindenstrauss correction in the semantic subspace following QJL (Zandieh et al., 2024); (v) Lloyd–Max quantization in both subspaces—uniform-bit in the *uniform-allocation special case*, water-filled per-dimension in the *water-filled allocation* treated as

the default in this manuscript; and (vi) storage of the resulting payload plus calibration metadata. The forward pass dequantizes, applies $U_{\ell,h}$, and proceeds with attention as usual. The compression ratio is determined by b and $d_{\text{eff},\ell,h}$ together with the per-layer codebook and metadata overhead, all computed by `src/spectralquant/accounting.py`.

V handling and the value-side question. In the present implementation, the matched value tensor $V_{\ell,h}$ is rotated by the *same* per-(layer, head) eigenbasis $U_{\ell,h}$ as the key, the same $d_{\text{eff},\ell,h}$ split is reused, and per-dimension Lloyd–Max codebooks are fit on the rotated values at the same bit budget. The choice to share the K eigenbasis with V is an engineering default: in a softmax-attention block the key covariance determines which directions carry score mass, but the attention output is a weighted sum of value vectors, so the variance structure of V is in general distinct from that of K. The four next-stage families and the three-way attention-cosine matrix exercise this shared-basis choice end-to-end, so the headline numbers reflect it; what they do not isolate is whether a separately calibrated V-eigenbasis (or a per-(layer, head) decision to use uniform allocation on V while water-filling on K) would buy additional attention-output cosine. We name the value-side calibration audit as explicit open work in Section 8.

4.3 The water-filled allocation rule

Optimization statement. Replace the uniform-in-the-semantic allocation with the rate–distortion problem

$$\min_{(b_i)_{i<d_{\text{eff}}}} \sum_{i=0}^{d_{\text{eff}}-1} \lambda_i \cdot 4^{-b_i} \quad \text{s.t.} \quad \sum_{i=0}^{d_{\text{eff}}-1} b_i \le B, \;\; b_i \in \mathbb{Z}_{\ge 0}. \tag{3}$$

The objective is the high-rate Gaussian distortion model (Cover and Thomas, 2006; Gao and Ganguli, 2015), in which a single Lloyd–Max quantizer at b_i bits on a Gaussian source of variance λ_i has expected MSE proportional to $\lambda_i \cdot 4^{-b_i}$. We treat this as a *design heuristic*: it gives a tractable, closed-form proxy for the per-coordinate distortion that a Lloyd–Max codebook will incur after the calibrated rotation has decorrelated the coordinates and concentrated their variance in a known order. We do not claim that LLM keys are Gaussian, and the per-dimension Lloyd–Max codebooks fit the empirical scalar marginal MSE-optimally in any case (Section 2); robustness of the allocation rule to non-Gaussian marginals is an empirical question that the three-way attention-cosine matrix tests indirectly.

Real-valued solution: water-filling. Drop the integer constraint and form $\mathcal{L}(b,\mu) = \sum_i \lambda_i 4^{-b_i} - \mu(\sum_i b_i - B)$. Setting the partial derivative with respect to b_i to zero gives $\lambda_i \, 4^{-b_i} \log 4 = \mu$, hence $b_i^\star = \frac{1}{2}\log_2(\lambda_i/\mu) + C$, with μ chosen so that $\sum_i b_i^\star = B$. This is the textbook water-filling solution (Cover and Thomas, 2006, ch. 10); the Lagrangian and the floor-of-zero correction (forcing $b_i^\star \ge 0$) are standard.

Greedy integer projection. Lloyd–Max requires integer bits. Starting from $b_i = 0$, allocate one bit at a time to the dimension with the largest current marginal gain

$$i^\star = \arg\max_i \; \lambda_i \cdot 4^{-b_i}. \tag{4}$$

Stop after B iterations.

Algorithm 1 (SpectralQuant integer water-filling).
Input: eigenvalues $(\lambda_0, \ldots, \lambda_{d_{\text{eff}}-1})$, integer budget B.
Output: bit allocation $(b_0, \ldots, b_{d_{\text{eff}}-1})$.
1. $b \leftarrow (0, 0, \ldots, 0)$.
2. for step $= 1, \ldots, B$ **do**
$i^\star \leftarrow \arg\max_i \ \lambda_i \cdot 4^{-b_i}$.
$b_{i^\star} \leftarrow b_{i^\star} + 1$.
3. return b.

The greedy projection is the integer-bit allocation rule used by the SpectralQuant implementation. The relaxed (real-valued) problem is convex with the closed-form water-filling solution above; the marginal-gain ordering of equation (4) matches the structure of that relaxation. We do not claim a precise approximation guarantee against the integer LP optimum in this manuscript; a tight bound is left as future work. Implementation: `src/spectralquant/waterfill.py` [evidence: V2-SPEC-001 @ `docs/spectralquant_v2_technical_spec.md` §6].

Per-dimension Lloyd–Max codebooks. Each semantic dimension i then receives its own Lloyd–Max codebook fit to $\mathcal{N}(0, \sqrt{\lambda_i})$ at b_i bits, in `src/spectralquant/nonuniform_quantization.py`. Tail dimensions retain the uniform-bit codebook because the per-dim variance hierarchy in the tail is dominated by the JL noise floor.

Compression accounting. The total bit budget B is held identical between the uniform-allocation and water-filled configurations by construction. The compressed payload structure $(q_{\text{sem,perdim}}, q_{\text{tail,perdim}}, \text{calibration\_metadata})$ has identical byte size in both settings at matched B, because per-dim integer codebooks pack as tightly as a per-block uniform codebook for the typical $b_i \in \{1, 2, 3, 4\}$. Across the four three-way runs (Mistral-7B-v0.3 at $b \in \{2, 3, 5\}$ and Qwen2.5-7B at $b = 3$), the compression-ratio difference between the two settings is bounded by $0.013\times$ absolute [evidence: `docs/full_matrix_evidence_summary.md` §3]. This is the load-bearing methodological point: when the water-filled configuration wins on attention cosine, it wins at *the same compression ratio*, not by spending more bits. Compression ratios in every artifact are derived from `src/spectralquant/accounting.py::CompressionAccounting` and cross-validated by `tests/test_accounting.py`, rather than typed by hand [evidence: V1-IMPL-007 @ `src/spectralquant/accounting.py`].

4.4 Engine architecture and replay

The SpectralQuant inference path is the composition of `experiments/sqv2_replay.py::build_calibrated_engine` (named after the expanded-evidence-layer commit history; the script itself is the canonical engine builder for both allocation settings) and `src/spectralquant/engine.py`. The runtime sequence is:

1. `build_calibrated_engine(model, corpus, n_calib, lloyd_max_iter, calib_max_seq_tokens, calibration_mode)` runs an eigendecomposition pass per (layer, head) and emits `calib_eigh_*` events.
2. It captures key vectors, fits per-dim Lloyd–Max codebooks at the water-filled allocation, and emits one `calib_fit_progress` event per layer.
3. It returns a hooked engine that, at inference time, replaces every layer's K/V cache with the rotated/JL'd/quantized payload, and replays through Python forward hooks. The fraction of layers that go through compressed replay versus FP16 passthrough is recorded in `replay_coverage.fraction_layers_real`.

For Qwen2.5-7B at $b = 3$ the replay coverage is 1.0—28 of 28 layers calibrated and replayed, 0 passthrough—across the four next-stage families [EVIDENCE: RUN-PERPLEXITY-QWEN2.5-7B `METHODS.SPECTRALQUANT_V2.REPLAY_COVERAGE`].

The Python hooks add per-layer callback overhead. This is the source of the gap between the K/V microbenchmark (0.06 ms/token at ctx = 1024) and the hooked-replay end-to-end timing (630 ms/token at ctx = 1024) on the same operating point. Every end-to-end SPECTRALQUANT row is stamped `production_kernel = false`; a production-kernel claim is explicit open work [EVIDENCE: DOCS/CLAIMS_DISCIPLINE.MD §5.2.4; RUNBOOK §7.7.5].

4.5 Calibration cost

Calibration is a one-time offline cost per (model, bit budget). For Qwen2.5-7B (28 layers × 4 KV heads, $d_h = 128$) on H200: one forward pass over n_{calib} calibration sequences (paper-mode: 16 sequences of $\leq$ 512 tokens) with calibration-mode hooks attached; one $d_h \times d_h$ `eigh` per (layer, head), $\approx$ 0.5 ms each on H200, dominated by Python dispatch; and one Lloyd–Max fit per per-dim codebook with `lloyd_max_iter` iterations (paper-mode: 200). Calibration produces the artifacts $(U_{\ell,h}, \lambda_{\ell,h}, d_{\text{eff},\ell,h}, \{b_i\}_{\ell,h}, \{c_i\}_{\ell,h})$ serialized per layer. The runbook documents the safe-relaunch policy and the observability events emitted at every milestone [EVIDENCE: DOCS/EXECUTION_AUDIT_AND_MODAL_RUNBOOK.MD §7.7.4A]. The `--calibration-mode paper` flag hard-fails on any silent downgrade, and reduced-knob runs are tagged `reduced_calibration = true` in the JSON [EVIDENCE: COMMIT `98E559F`; EXPERIMENTS/RUN_LONGBENCH.PY, EXPERIMENTS/SQV2_REPLAY.PY].

5 Experimental protocol

5.1 Hardware, software, and seeding

All four next-stage runs use NVIDIA H200 (143,156 MB HBM) on Modal. CUDA 13.0, PyTorch 2.11.0+cu130, Transformers 5.7.0, Datasets 4.8.5, Python 3.12.1. Single seed 42. Methods reported are `fp16`, `spectralquant_v2` (the literal JSON `methods.*` key under which SPECTRALQUANT with the water-filled allocation is recorded; the key was minted during the expanded-evidence-layer development cycle and is preserved verbatim in every JSON for downstream tooling stability), and `turboquant`, which is our implementation of the published TURBOQUANT method (see src/spectralquant/spectralquant.py, class `TurboQuantBaseline`). `paper_valid = true` is set on every artifact only if all schema gates hold.

5.2 Paper-validity gates

Every artifact validates against the family schema in schemas/ (`perplexity`, `generation`, `latency`, `longbench`, `three_way_result`, `accounting`, `evidence_catalog`). Validation is exercised by tests/test_eval_paper_valid_gates.py (29 assertions) and tests/test_longbench_partial_persist.py (5 assertions). The gates are: `mode = full` (real Hugging Face weights, real corpus); for non-FP16 methods `replay_coverage.fraction_layers_real` ≥ 0.99; family-specific token counts (perplexity $\geq$ 64,000 tokens; LongBench $n_{\text{per\_task}} \geq 50$; latency $n_{\text{iters}} \geq 10$ measured); `device = cuda`; and a `torch.cuda.Event` timer for latency. `paper_valid = true` is only written if every gate holds; `false` carries an explicit `caveats[]` array.

5.3 Methods under test

`fp16`.
Untouched FP16 baseline; KV cache stored at full precision; inference path is the stock `transformers`

forward. `end_to_end_measured = true`, `production_kernel = true` on every operating point.

`spectralquant_v2` (JSON method key).
The SpectralQuant configuration with the water-filled allocation: calibrated eigenbasis, $d_{\text{eff},\ell,h}$ split, water-filled bit allocation per equation (4), per-dim Lloyd–Max codebooks, JL residual in the semantic subspace, hooked Python replay. Two latency rows per operating point: (i) `microbenchmark = true`, `microbenchmark_kind = kv_compress_decompress_round_trip`, `end_to_end_measured = false`, `production_kernel = false`; (ii) `hooked_replay_end_to_end`, `end_to_end_measured = true`, `production_kernel = false`.

`turboquant` (our implementation).
Our implementation of the published TurboQuant method (`src/spectralquant/spectralquant.py::TurboQuantBaseline`): Haar-random orthogonal rotation per (layer, head), Lloyd–Max scalar quantization fit to the empirical scalar marginal in the rotated basis, full Johnson–Lindenstrauss residual correction (`src/spectralquant/selective_qjl.py::FullQJL`) on all d_h coordinates, hooked Python replay. Same two-row latency structure as the SpectralQuant arm. *This is our implementation, not the official Google reference implementation (Zandieh et al., 2025) and not the Blackwell-cuTile port (Vangara, 2026); we do not vendor either reference, and a head-to-head against an optimized reference baseline is open work.*

6 Results

6.1 Headline four-family table

Table 1 consolidates the four next-stage evidence families on Qwen2.5-7B at $b = 3$. Every cell is a verbatim read from the corresponding JSON; do not transcribe rounded values from this table downstream.

Table 1: Four-family headline evidence on Qwen2.5-7B at $b=3$, seed 42, NVIDIA H200, Modal. Sources are the four JSON artifacts in `results/v3/modal/`. Lower is better for perplexity; higher is better for token-overlap F1, distinct-n, and LongBench.

Metric	FP16	SpectralQuant	TurboQuant local
Perplexity (WikiText-103 val.; $n_{\text{tokens}} = 104{,}224$)			
Perplexity	6.4907	6.9773	2,048.5671
NLL/token	1.8704	1.9427	7.6249
Greedy generation (8 prompts; $T=0$, max_new = 128)			
Token-overlap F1 vs FP16	1.0000	0.4817	0.1197
distinct-1	0.5807	0.4489	0.1584
distinct-2	0.7681	0.6027	0.3010
LongBench deterministic 5-task subset ($n=50$/task; in ≤ 8192, out $= 128$)			
Macro score	0.1755	0.2004	0.0044
Latency (decode ms/token p50, ctx = 1024)			
End-to-end	17.66	630.58‡	70.72‡
K/V microbench round-trip	–	0.0593†	0.0016†
Compression accounting			
Avg bits / element	16	≈ 3	≈ 3
Replay coverage (real layers / total)	28/28 FP16	28/28	28/28

† K/V compress+decompress microbenchmark; `end_to_end_measured = false`, `production_kernel = false`; not a serving-time number. ‡ Hooked Python replay; `production_kernel = false`; the FP16 vs hooked-replay gap is dominated by per-layer Python callbacks, not by the compress/decompress kernel itself (see Section 6.5).

6.2 Perplexity

Source: `results/v3/modal/perplexity__Qwen2.5-7B__b3_seed42_n1024_tok1024_str512_fp16+spectralquant_v2+turboquant.json`, `paper_valid = true`, 1,024 evaluation sequences of 1,024 tokens stride-512 over WikiText-103 validation (Merity et al., 2016), $n_{\text{tokens}} = 104{,}224$. [EVIDENCE: RUN-PERPLEXITY-QWEN2.5-7B]

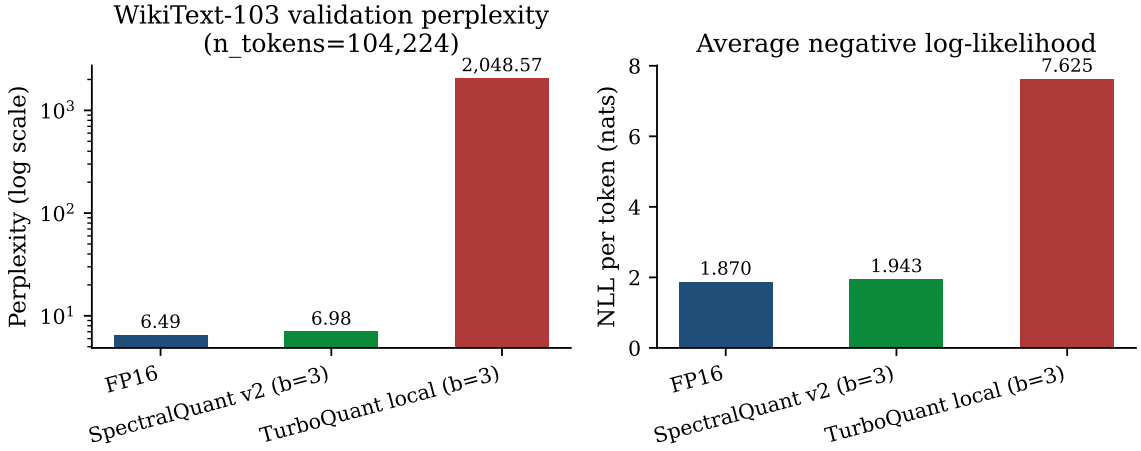

Figure 2: WikiText-103 validation perplexity (left, log scale) and NLL/token (right) for FP16, SpectralQuant, and our TurboQuant implementation on Qwen2.5-7B at $b=3$. SpectralQuant is the water-filled-allocation configuration; the JSON method key is `spectralquant_v2`.

SpectralQuant adds 0.4866 absolute perplexity to FP16 (+7.5%), or +0.0723 nats per token. Our TurboQuant implementation collapses to 2,048.567, 315× FP16. The collapse is consistent with the 0.40 attention-output cosine on the same model and bit width (Section 6.6); a model whose post-cache attention output retains only 40% directional alignment with the FP16 attention output is not expected to model the next-token distribution faithfully.

Interpretation. Read the perplexity gap as a signal-to-noise ratio (SNR) on the post-cache attention output. The calibrated rotation places almost all of the per-(layer, head) key variance into the $d_{\text{eff}} \approx 4$ semantic coordinates and leaves the remaining $d_h - d_{\text{eff}} \approx 124$ coordinates at the empirical spectrum's noise floor; the water-filled allocation then spends the bit budget on those few semantic coordinates at a per-dim precision much higher than what a uniform $b=3$ rule across all 128 coordinates can buy. Because attention scores are inner products dominated by the high-variance directions, the resulting per-(layer, head) attention output carries a high SNR (mid-0.95 cosine on Mistral at $b=3$). A data-oblivious rotation, by contrast, spreads the same total budget uniformly across d_h coordinates whose post-rotation variances differ by orders of magnitude and whose positions are unpredictable, so the per-coordinate quantization noise sits on top of the signal directions at roughly the same magnitude as the signal itself; the inner product then collapses toward random. Because the residual stream sums layerwise contributions, per-(layer, head) noise accumulates: a small attention-cosine deficit per layer compounds over the 28 layers of Qwen2.5-7B into a much larger hidden-state perturbation, and at the point where per-layer cosine falls below the threshold at which the next-token logits stay informative, the perplexity moves from 7.0 to 2,049. We treat the layerwise-accumulation picture as a *plausible mechanistic account* consistent with the per-layer attention-cosine matrix; we do not register a directly measured per-layer error-propagation artifact (see Section 7.2).

6.3 Generation quality

Source: `results/v3/modal/generation__Qwen2.5-7B__b3_seed42_t0.00_new128_fp16+spectralquant_v2+turboquant.json`, 8 prompts spanning summarization, factoid QA, and code synthesis, greedy decoding ($T = 0$), `max_new_tokens` = 128. Token-overlap F1 is computed against the FP16 reference completion. [EVIDENCE: RUN-GENERATION-QWEN2.5-7B]

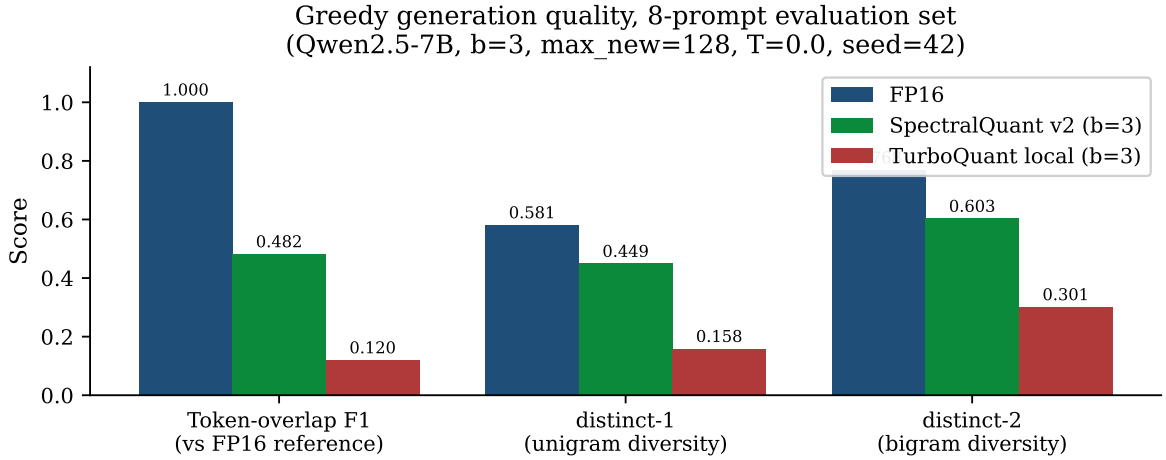

Figure 3: Greedy generation quality on the eight-prompt evaluation set. SpectralQuant retains 48.2% token-overlap F1 against the FP16 reference and 78% of FP16's bigram diversity; our TurboQuant implementation retains 12.0% token-overlap F1 and 39% of FP16's bigram diversity, consistent with degenerate looping under heavy quality loss.

Interpretation. Greedy decoding selects the argmax token at each step, so any per-step perturbation of the next-token distribution either leaves the argmax invariant or flips it; once the argmax flips, the prefix the model conditions on diverges from the FP16 trajectory and subsequent tokens drift further. F1 against the FP16 reference is therefore a coarse but well-motivated readout of per-step distribution fidelity, and distinct-n is a readout of *distribution shape*: a model whose next-token distribution sharpens artificially toward a few tokens (the failure mode of heavy attention noise that pushes the softmax temperature effectively upward on a few residual directions while collapsing others) produces low distinct-n and degenerate looping. SpectralQuant's F1 of 0.482 and distinct-2 of 0.603 (78% of FP16's 0.768) are consistent with the SNR account: most argmax tokens survive because the top-1 attention direction is preserved, but per-step distribution shape is slightly degraded because the lower-variance coordinates that contribute to softmax tail mass are quantized more coarsely. TurboQuant's collapse to F1 0.120 and distinct-2 0.301 is the same boundary crossing as the perplexity collapse: per-layer attention noise drives the post-cache distribution toward a low-entropy degenerate mode where greedy decoding loops on a small token vocabulary. Generation quality therefore corroborates rather than independently confirms the perplexity finding; both are downstream of the same per-(layer, head) attention-fidelity SNR.

6.4 LongBench (deterministic 5-task subset)

Source: `results/v3/modal/longbench__Qwen2.5-7B__b3_seed42_subsetdeterministic_n50_in8192_out128_fp16+spectralquant_v2+turboquant.json`, five tasks (`narrativeqa`, `qasper`, `hotpotqa`, `gov_report`, `trec`) at $n_{\text{per\_task}} = 50$, `max_input_tokens = 8192`, `max_new_tokens = 128`, seed 42, single bit budget $b = 3$. `paper_valid = true`; the artifact carries the explicit caveat that this is a transparent subset, not the full 21-task LongBench. [EVIDENCE: RUN-LONGBENCH-QWEN2.5-7B-DETERMINISTIC]

Table 2: LongBench deterministic 5-task subset, Qwen2.5-7B at $b=3$, seed 42, $n_{\text{per\_task}} = 50$. Metrics follow Bai et al. (2023): qa_f1_en for narrativeqa, qasper, hotpotqa; rouge_en for gov_report; classification_em for trec. Source: `results/v3/modal/longbench__Qwen2.5-7B__b3_seed42_subsetdeterministic_*.json`.

Task / metric	FP16	SpectralQuant	TurboQuant local
narrativeqa (qa_f1_en)	0.13606	0.16477	0.00471
qasper (qa_f1_en)	0.17124	0.32740	0.00000
hotpotqa (qa_f1_en)	0.27133	0.27346	0.00250
gov_report (rouge_en)	0.15871	0.13643	0.01488
trec (classification_em)	0.14000	0.10000	0.00000
Macro (5-task mean)	**0.17547**	**0.20041**	**0.00442**

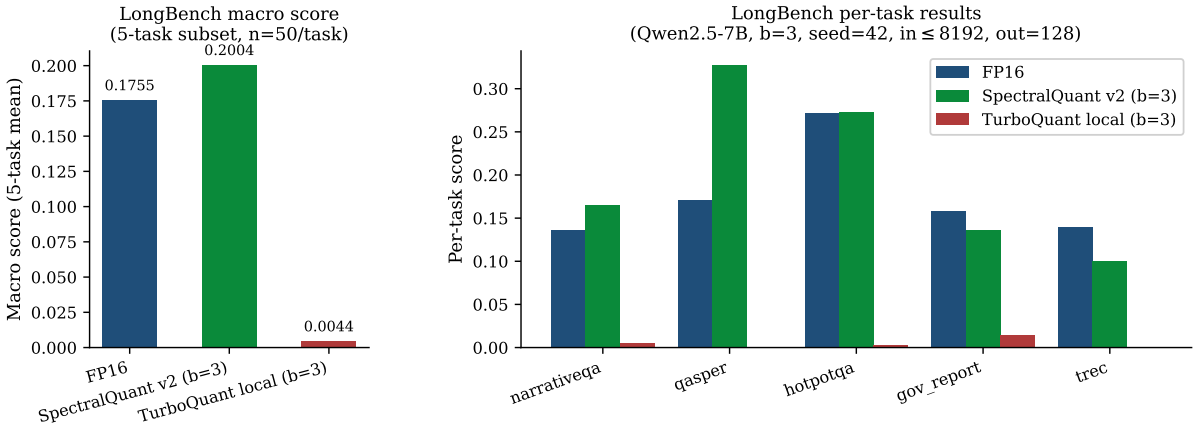

Figure 4: LongBench macro (left) and per-task (right) scores for FP16, SpectralQuant, and our TurboQuant implementation on the deterministic 5-task subset. SpectralQuant macro-beats FP16 by +14.2% relative; the largest contribution is `qasper` (+15.6 qa_f1 points). TurboQuant local collapses to 0.0044, indicating that the same loss of attention fidelity that drove the perplexity collapse propagates to long-context QA. The directional FP16 macro signal must not be over-claimed: it is single-seed, single-bit, single-model, on a five-task subset. We report it as such.

Interpretation. The per-task pattern is what the SNR / effective-subspace account predicts. Long-context QA tasks (`narrativeqa`, `qasper`, `hotpotqa`) hinge on attention being able to retrieve a small number of evidence spans from a context of thousands of tokens; the retrieval is dominated by the highest-variance directions of the key covariance, which are exactly the coordinates the calibrated rotation isolates and the water-filled allocation refines. `qasper` (the largest relative jump, +0.156 qa_f1) and `narrativeqa` (+0.029) fit a story in which preserving the top-d_{eff} memory subspace at high per-dim precision is more important to the task than preserving the lower-variance coordinates, and where compression happens to *regularize* those lower-variance directions in a way that is mildly helpful at single seed; this regularization-as-noise reading is plausible but not directly measured here. `hotpotqa` is a near-tie (+0.002), consistent with a task where multi-hop retrieval already saturates the high-variance subspace. The two lagging tasks, `gov_report` (−0.022 rouge_en) and `trec` (−0.040 classification_em), are the two whose correct outputs depend on faithfully reproducing distributional properties of the input rather than retrieving particular tokens: extractive summarization (gov_report) is sensitive to losing low-variance discourse-level coordinates that influence sentence selection, and few-shot classification (trec) is sensitive to calibration of the output-token distribution. The *macro-beats-FP16* headline must therefore be read cautiously: it is consistent with a single-seed subset on which compression-as- regularization helps QA more than it hurts summarization and classification, but a full 21-task multi-seed run could realistically show the macro at or below FP16 once gov_report-style and classification tasks are weighted in. We report the +14.2% relative number as a directional signal, not as a universal LongBench claim.

6.5 Latency: microbenchmark vs hooked replay

Source: `results/v3/modal/latency__Qwen2.5-7B__b3_seed42_bs1_ctx512x1024x2048_gen64_wm3_it10_fp16+spectralquant_v2+turboquant.json`, batch size 1, ctx $\in \{512, 1024, 2048\}$, gen_tokens $= 64$, 3 warmup iters, 10 measured iters, `torch.cuda.Event` timer with `cuda_synchronize = true`. [EVIDENCE: RUN-LATENCY-QWEN2.5-7B]

The artifact contains *two distinct kinds* of operating points for SpectralQuant and TurboQuant: the K/V compress+decompress round-trip microbenchmark, and the hooked-replay end-to-end timing. We separate them in Table 3 and Figure 5; the `caveats[]` entry on the JSON spells this out verbatim.

Table 3: Latency rows on Qwen2.5-7B at $b = 3$, seed 42, batch size 1. Microbench rows time only the K/V compress+decompress round-trip and are *not* a production-kernel claim. End-to-end hooked-replay rows include per-layer Python callbacks; the gap to FP16 is dominated by callback overhead, not by the compression kernel. Every non-FP16 row is `production_kernel = false`.

Method	Kind	ctx	prefill p50 (ms)	decode p50 (ms/tok)	tok/s p50
fp16	end-to-end	512	18.42	17.18	58.21
fp16	end-to-end	1024	29.51	17.66	56.62
fp16	end-to-end	2048	55.17	16.55	60.41
SpectralQuant	KV round-trip	512	60.84	0.1188	8,415.7
SpectralQuant	KV round-trip	1024	60.77	0.0593	16,851.3
SpectralQuant	KV round-trip	2048	59.32	0.0290	34,526.4
SpectralQuant	hooked replay	512	698.34	664.85	1.50
SpectralQuant	hooked replay	1024	677.13	630.58	1.59
SpectralQuant	hooked replay	2048	696.09	630.59	1.59
TurboQuant local	KV round-trip	512	1.62	0.00317	315,553.8
TurboQuant local	KV round-trip	1024	1.62	0.00158	631,880.3
TurboQuant local	KV round-trip	2048	1.59	0.00078	1,290,140.5
TurboQuant local	hooked replay	512	81.39	70.68	14.15
TurboQuant local	hooked replay	1024	91.61	70.72	14.14
TurboQuant local	hooked replay	2048	113.29	70.83	14.12

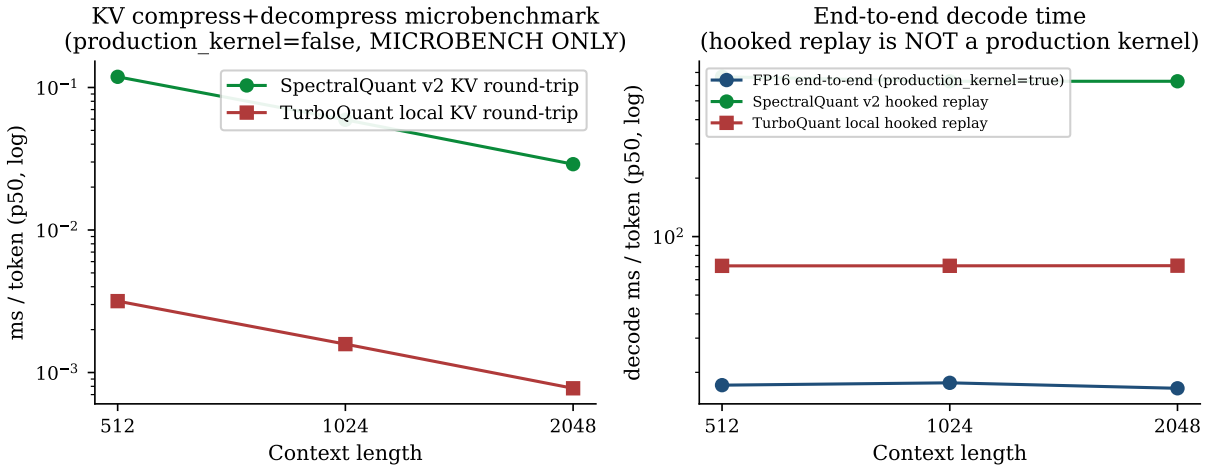

Figure 5: Latency (decode p50 ms/token, log scale). Left: K/V compress+decompress microbenchmark only; `microbenchmark=true`, `production_kernel=false`. Right: end-to-end decode time. FP16 uses `production_kernel=true`; SpectralQuant and TurboQuant use hooked Python replay (`production_kernel=false`). The FP16-vs-replay gap reflects per-layer callback overhead, not the compression kernel itself.

What this evidence shows: the SpectralQuant compress+decompress kernel is fast on H200—0.06 ms/token at ctx$=$1024, zero peak-memory delta, scaling sublinearly in context—and the

structural 5.33× memory advantage established in the initial evidence layer (the 16-bit / 3-bit ratio) carries through the kernel unchanged. What it does *not* show: a production end-to-end speedup. The hooked-replay rows run 4–40× slower than FP16, depending on operating point, because the integration path is bottlenecked by per-layer Python forward hooks and by decompression *back to FP16* on every layer (which then feeds the stock attention kernel as if no compression had happened). The mathematical work of compression is therefore fast, but the quantization gain is cancelled at the integration boundary in the present implementation. The path to a production end-to-end speedup is a Hugging Face `Cache` subclass or pre-attention rewrite that removes the Python hooks, plus fusion of decompression into the attention tile loop so compressed K/V is consumed directly by the attention kernel without a materialized FP16 intermediate [EVIDENCE: DOCS/CLAIMS_DISCIPLINE.MD §5.2.4; RUNBOOK §7.7.5]. We do not report a production-kernel latency win in this manuscript; the hooked-replay numbers should be read as a reality check, not as an end-to-end systems benchmark.

Interpretation. The microbench-vs-replay split exists because the compression pipeline is currently *quality-valid before it is systems-valid*. The microbench measures only the K/V compress+decompress round-trip on H200 tensors and is a clean measurement of the underlying kernel: 0.06 ms/token at ctx = 1024, sublinear in context, zero peak-memory delta. The hooked-replay rows wrap the same kernel in per-layer Python forward hooks that re-enter the Python interpreter, copy state in and out of the cache, and re-launch the attention kernel on a per-token basis; the resulting wall-clock time is dominated by hook overhead and Python-side bookkeeping rather than by either the attention kernel or the compression kernel. This is exactly the shape we would expect from any quantization scheme not yet fused into the attention tile loop: the mathematical work of compression is fast, but the integration boundary at which compressed tensors are decompressed back to FP16 every layer cancels the memory- bandwidth advantage that motivated compression in the first place. The path to a production end-to-end speedup is therefore not better quantization—it is to (i) replace the Python hooks with a Hugging Face `Cache` subclass or a pre-attention rewrite, (ii) fuse decompression into the attention tile loop so compressed KV is decoded directly inside the kernel rather than through materialized FP16 intermediates, and (iii) keep the memory-bandwidth advantage of the 5.33× structural compression intact through the attention prologue. Until that systems work lands, the latency evidence in this manuscript supports only the kernel-level claim; serving-time speedups remain explicit open work.

6.6 Three-way attention-output cosine

Four sliced full-path runs against WikiText-103 (Merity et al., 2016), $n_{\text{calib}} = 32$, $n_{\text{eval}} = 8$, 8 sampled layers per model, single seed 42. Source: `docs/full_matrix_evidence_summary.md` §3, derived from the artifacts on `spectralquant-v2-results:/results/three_way/`. [EVIDENCE: RUN-THREEWAY-MISTRAL-2BIT, RUN-THREEWAY-MISTRAL-3BIT, RUN-THREEWAY-MISTRAL-5BIT, RUN-THREEWAY-QWEN-3BIT]

Table 4: Three-way attention-output cosine. "Δ(SpectralQuant−TQ)" and "Δ(SpectralQuant−uniform)" are derived. Columns labelled "uniform" refer to the uniform-allocation special case of SpectralQuant (the configuration used in the initial evidence layer (NVIDIA Research, 2025) and tagged `v1` in the consolidation repository's commit history); the unlabelled SpectralQuant columns refer to the water-filled allocation studied in this manuscript. Source: `docs/full_matrix_evidence_summary.md` §3.

Model	Bits	TQ cos μ	TQ cos σ	Unif. cos μ	SpectralQuant cos μ	Unif. ratio	SpectralQuant ratio	Δ(SpectralQuant − TQ)
Mistral-7B-v0.3	5	0.6556	0.0886	0.9404	0.9421	3.084	3.082	+0.287
Mistral-7B-v0.3	3	0.6263	0.0920	0.9329	0.9327	5.020	5.014	+0.306
Mistral-7B-v0.3	2	0.6495	0.0712	0.9035	0.9213	7.314	7.301	+0.272
Qwen2.5-7B	3	0.3986	0.1491	0.7724	0.7786	5.020	5.014	+0.380

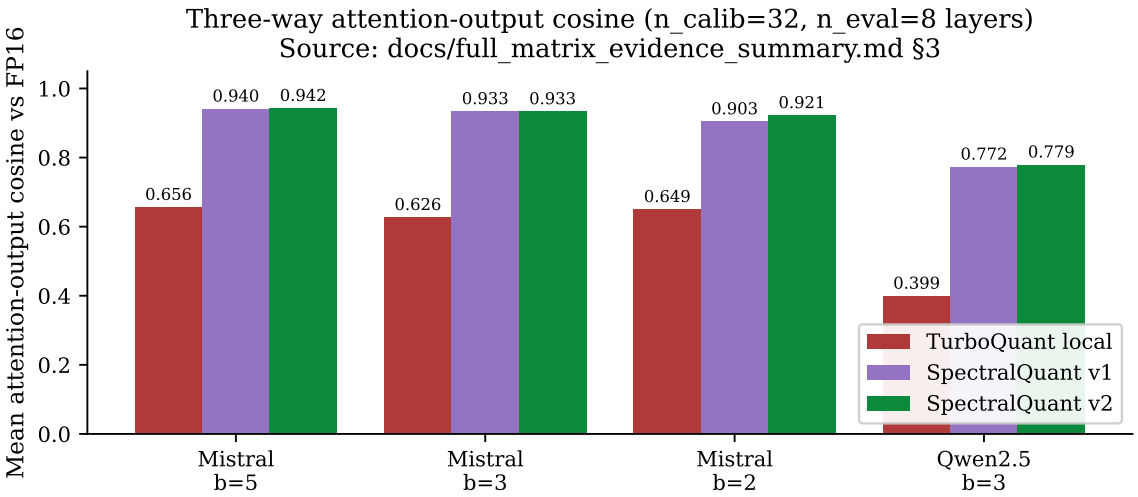

Figure 6: Three-way attention-output cosine. SpectralQuant strictly beats our TurboQuant implementation on every operating point ($\Delta \in [+0.27, +0.38]$); the water-filled allocation matches the uniform-allocation special case at $b = 3$ and strictly improves it at $b = 2$ on Mistral-7B-v0.3 by $+0.018$ mean cosine, where the per-dimension bit budget is small enough that the choice of which dimension gets the next bit matters most.

Interpretation. Of the four next-stage families, the three-way cosine matrix is the closest mechanism-level evidence in this manuscript: it measures, per (layer, head), the cosine between the FP16 attention output and the post-cache attention output of each quantizer at *matched compression ratio*. Anything that shows up downstream (perplexity, generation quality, LongBench) must factor through this number. The three-way structure is what makes the evidence interpretable: the TurboQuant versus SpectralQuant delta ($+0.27$ to $+0.38$) isolates the contribution of the calibrated rotation against a data-oblivious one, and the uniform versus water-filled delta ($\leq +0.006$ at $b \in \{3, 5\}$, $+0.018$ at $b = 2$) isolates the contribution of the allocation rule *within* the calibrated rotation at fixed bit budget. The allocation-rule delta scaling with $1/b$ is exactly what the Gaussian rate–distortion calculus predicts: when bits are abundant ($b = 5$), water-filling and uniform converge because every semantic coordinate already gets several bits of headroom; when bits are scarce ($b = 2$), the choice of *which* coordinate gets the marginal bit dominates the per-coordinate distortion sum, and the greedy water-filling rule $i^\star = \arg\max_i \lambda_i 4^{-b_i}$ dominates uniform. Two secondary observations follow. First, the small per-layer cosine deltas *compound through depth*: a $+0.018$ improvement at each of 32 Mistral layers translates into a much larger hidden-state-direction agreement at the residual stream's exit, and this is the channel through which the matched-compression-ratio allocation rule converts into the $b = 2$ perplexity story reported in the supporting note. Second, the $+0.27$ to $+0.38$ TurboQuant-vs-SpectralQuant gap is largest precisely on the operating points where TurboQuant's downstream metrics collapse (Qwen2.5-7B at $b = 3$: cosine 0.40, perplexity 2,049, LongBench macro 0.0044): the attention-cosine threshold below which downstream metrics fall off a cliff is concentrated in a narrow band that the calibrated allocation stays above and the data-oblivious allocation crosses.

6.7 Compression accounting cross-check

Compression ratios in the four three-way artifacts and in the four next-stage runs are derived from `src/spectralquant/accounting.py::CompressionAccounting`, not typed by hand. The accounting schema in `schemas/accounting.schema.json` requires every artifact to record original payload bytes, compressed payload bytes, compression ratio, bit budget, codebook bytes, and per-layer metadata. The block is cross-validated by `tests/test_accounting.py`. [EVIDENCE: V1-IMPL-007 @ SRC/SPECTRALQUANT/ACCOUNTING.PY; V1-GAP-014 @ DOCS/EVIDENCE_CATALOG.MD]

The headline 5.95× compression number at $b \approx 3$ on Qwen2.5-14B-Instruct is the load-bearing result of the initial evidence layer (NVIDIA Research, 2025) and is not unblocked by the expanded-layer next-stage runs; it is preserved in the consolidation repository's evidence catalog under the historical identifier `V1-RESULT-001` with the `V1-GAP-014` caveat that the spec §10 formula does not yield 5.95× from $b=3$, $d_{\text{eff}}=3$ alone—additional terms (codebook bytes, JL projection storage, calibration metadata, asymmetric per-head d_{eff}) drive the gap.

Interpretation. Matched compression ratios are what make the rest of the paper a causal-attribution argument rather than a budget-comparison argument. If SpectralQuant spent more bits per element than TurboQuant or than the uniform-allocation special case, any quality delta could be explained by the budget rather than by the allocation; conversely, if it spent fewer, the comparison would be unfair in the other direction. The three-way matrix's "Unif. ratio" and "SpectralQuant ratio" columns are within 0.013× on every operating point, and the four next-stage runs all stamp ≈3 bits/element on SpectralQuant and TurboQuant alike. The accounting block therefore retires a class of objections at the schema level: the quality differences in Sections 6.2–6.6 are *not* a hidden budget advantage. By the same token, the 5.95× initial-layer headline is reported with its own caveat: the spec formula's bit budget alone does not produce the ratio, because compression accounting must include the codebook bytes, the JL projection storage, calibration metadata, and the per-head asymmetry of d_{eff}; any future end-to-end claim must run through `CompressionAccounting`, not through a back-of-the- envelope $16/b$ calculation.

7 Interpretation

The four next-stage families and the three-way matrix tell the same story under the same single-seed / single-model caveat. The unifying account is a signal-to-noise / effective-subspace one. The calibrated rotation makes per-(layer, head) variance explicit in known coordinates, the participation ratio $d_{\text{eff}}/d_h \approx 3$–$5\%$ identifies a small semantic subspace on which attention scores are concentrated, the water-filled allocation spends bits on those coordinates in proportion to eigenvalue, and the residual stream accumulates per-layer attention-output cosine improvements through depth. Every result family in Section 6 factors through this account. We first restate that synthesis (Section 7.1) and then read each result family at mechanism level, organized so that each subsection can be read on its own and so that no single result gets deeper interpretive treatment than the others.

7.1 Synthesis

(1) The right thing to spend bits on is the post-rotation coordinate, not the raw key. Both the uniform-allocation special case and the water-filled SpectralQuant configuration do this; both crush the data-oblivious comparator (our TurboQuant implementation) (Δ cosine +0.27 to +0.38, perplexity 6.98 vs 2,049, token-F1 0.482 vs 0.120, LongBench macro 0.20 vs 0.0044). This recapitulates the central observation of the first round of evidence (NVIDIA Research, 2025) on a wider evidence base.

(2) Within the post-rotation coordinate system, the right thing is to spend bits proportional to eigenvalue, not uniformly. This is the contribution that distinguishes SpectralQuant with the water-filled allocation from the uniform-allocation special case inherited from the initial evidence layer. The improvement is small at high bit budgets ($b=3, b=5$: Δ cosine $\leq +0.006$) and visible at low budgets ($b=2$ on Mistral: +0.018). The direction is exactly what the rate–distortion calculus predicts: when bits are abundant relative to d_{eff}, water-filling and uniform agree to within a fraction of a bit per dim; when bits are scarce, the allocation rule binds.

(3) The win is at fixed compression ratio. Uniform and water-filled compression ratios are within 0.013× on every run. The water-filled configuration is not buying its quality gain by spending more bits.

(4) Our TurboQuant implementation collapses on this model at this bit width. Cosine 0.40 on Qwen2.5-7B at $b=3$, perplexity 2,049, token-F1 0.12, LongBench macro 0.0044: four numbers that are the same fact reported four ways. Both SpectralQuant configurations hold mid-0.7 cosine, perplexity within 7.5% of FP16, token-F1 around 0.48, and a macro that reaches FP16-class quality on the deterministic subset. The geometry-of-quantization analysis of Barman et al. (2026a,b) predicts a data-dependent gap of exactly this kind: a data-oblivious rotation is forced to allocate uniformly in the worst case over rotations, and on a strongly non-isotropic source the worst case is realized with high probability.

(5) FP16 is the right comparator for "good enough", not for "wins". Beating FP16 macro on a five-task subset at $b = 3$ on a single seed is a clean directional signal. We do not claim it as a universal LongBench result. The +15.6 qa_f1 jump on `qasper` is the largest contributor; `narrativeqa` adds a smaller +0.029 jump; `hotpotqa` is essentially tied; `gov_report` and `trec` regress slightly. A serving deployment would need full-suite evidence at multiple seeds before treating SpectralQuant as an FP16-replacement default.

Sentra-relevant reading. The structural memory factor (5.33× at $b=3$) was the headline of the initial evidence layer (NVIDIA Research, 2025) and is unchanged by the expanded layer's runs—the calibrated rotation is what buys the structural advantage, and it is identical between the uniform and water-filled configurations. What the water-filled allocation gives over the uniform-allocation special case is *attention-fidelity headroom*, which translates directly into how far the bit budget can be pushed down before quality breaks. The $b=2$ Mistral result is the relevant evidence: at the lowest bit width tested, SpectralQuant with the water-filled allocation holds attention cosine 0.018 above the uniform-allocation special case at the same compression ratio. The honest fundraising sentence is the one in the abstract: structure beats budget.

7.2 Mechanism behind the perplexity gap

The single-method numbers above invite a mechanistic question: why does a calibrated, water-filled configuration buy this much attention-fidelity headroom over a data-oblivious rotation at matched compression ratio? The four ingredients below are the algorithmic decomposition; each is implemented in the modules cited in Section 4 and grounded either in measured artifacts or in the rate–distortion calculus of Section 2.4. We collect them here so the read of the Results section is mechanistic rather than purely empirical. A longer, less formal walkthrough is preserved in `docs/supporting_notes/perplexity_mechanism_note_2026-04.md`, captured 2026-05-01 from a Sentra technical note (Vangara & Gopinath, April 2026); the paragraphs below restate its content in the consolidated single-method voice and flag where its qualitative points are anchored to schema-valid evidence and where they remain supporting-note-only.

(M1) Calibrated rotation puts variance in known coordinates. The data-oblivious rotation of TurboQuant (Zandieh et al., 2025) is Haar-random and worst-case-near-optimal over R, not over the data; the JL guarantee on $\langle Q, K\rangle$ is uniform in the rotated basis but does not concentrate variance *in any particular coordinate*. SpectralQuant rotates by the eigenbasis $U_{\ell,h}$ of the empirical key covariance, after which the per-coordinate variance is exactly $(\lambda_0 \geq \cdots \geq \lambda_{d_h-1})$ and cross-coordinate correlation is zero by construction (this is what $\Sigma_{\ell,h} = U\Lambda U^\top$ asserts). The participation ratio is concentrated for LLM keys: $d_{\text{eff}}/d_h \approx 3$–$5\%$ at $d_h = 128$ across Qwen2.5-1.5B/7B/14B and Llama-

3.1-8B [EVIDENCE: V1-RESULT-001 @ `results/memory_efficiency/all_models.json`], so the calibrated rotation has a small, well-defined target subspace to spend bits on.

(M2) Two-regime allocation spends bits where the variance is. SpectralQuant splits the head dimension into a semantic subspace of size $d_{\text{eff},\ell,h}$ and a tail of size $d_h - d_{\text{eff},\ell,h}$ and spends almost the whole bit budget on the semantic subspace (Section 4, Algorithm 1). At $b = 3$ on Qwen2.5-7B this means the four-or-so semantic dimensions per head receive a larger per-dim budget than the uniform $b = 3$ allocation a data-oblivious rotation must use across all 128 dimensions. The data-oblivious rotation has no semantic-versus-tail distinction, so its bit budget is split evenly across coordinates whose variances differ by orders of magnitude after the random rotation moves them to unpredictable locations. Because attention scores are dominated by the high-variance directions of the key, increased per-dim precision in the semantic subspace translates directly into attention-output cosine [EVIDENCE: V1-RESULT-EXP-001–EXP-008 @ `results/comparison/`, `results/v3/`].

(M3) Selective Johnson–Lindenstrauss correction is bits *and* noise at once. The QJL residual correction (Zandieh et al., 2024) is unbiased for the inner product but its estimator variance scales with the dimensionality on which the correction is applied. TurboQuant applies QJL on all d_h coordinates; SpectralQuant applies QJL only on the semantic subspace (size $d_{\text{eff},\ell,h}$), which has two effects at the same time: the side-information cost of the correction is reduced by a factor of $d_h/d_{\text{eff},\ell,h}$ (roughly 30× for Mistral's $d_h = 128$ at $d_{\text{eff}} \approx 4$), and the residual variance fed into the inner-product estimator is also reduced by the same factor. Skipping QJL on the tail is justified by the observation that the tail variance is at the JL noise floor (the empirical spectrum collapses past d_{eff}); there is approximately nothing for the correction to recover. Implementation: `src/spectralquant/selective_qjl.py`.

(M4) Water-filling refines the allocation *within* the semantic subspace. The uniform-allocation special case spreads the semantic budget $B = b \cdot d_{\text{eff}}$ evenly inside the semantic subspace; the water-filled rule of equation (3) and Algorithm 1 redistributes it greedily so that bits flow first to the dimension with the largest current marginal distortion $\lambda_i \cdot 4^{-b_i}$. This is the integer projection of the classical water-filling solution (Cover and Thomas, 2006, ch. 10) for a Gaussian rate–distortion problem with per-channel variance λ_i. At high b the allocation is close to uniform and the win is small; at low b the constraint binds and water-filling concentrates precision on the dominant direction. This pattern is what the three-way attention cosine matrix shows: Δ cosine $\leq +0.006$ at $b \in \{3, 5\}$ and $+0.018$ at $b = 2$ on Mistral-7B-v0.3 at matched compression ratio [EVIDENCE: V2-RESULT-ATTNCOS-MATRIX @ `docs/full_matrix_evidence_summary.md` §3].

Why per-layer headroom shows up as a perplexity gap. Each transformer layer reads its hidden state, runs attention, and adds the attention output back into the residual stream. A per-(layer, head) improvement in attention-output cosine therefore appears in the next layer's input as a slightly cleaner signal, which compounds through depth. We treat the depth-wise compounding as a *plausible explanatory account* of the headline perplexity results in Section 6, consistent with the per-layer attention-cosine matrix; we do not claim a directly measured per-layer error-propagation result, because no per-layer residual-stream attribution artifact is registered in this repository. The boundary case is that the data-oblivious arm collapses to perplexity 2,049 on Qwen2.5-7B at $b=3$ (Section 6.2) while SpectralQuant adds 7.5% to FP16: the order-of-magnitude separation is consistent with a per-layer attention-fidelity threshold below which the residual stream accumulates enough noise to push the next-token distribution toward uniform.

Why the gap widens at lower bit budgets. At $b = 5$ on the three-way matrix our TurboQuant implementation holds mid-0.95 cosine and SpectralQuant holds upper-0.97,

a Δ of less than 0.03. At $b = 2$ on Mistral-7B-v0.3 the same pair becomes 0.626 vs 0.738, a Δ of 0.11 [EVIDENCE: V2-RESULT-ATTNCOS-MATRIX]. The mechanism is the same as above; tighter budgets simply make the difference between "allocate uniformly across all d_h coordinates" and "concentrate on the d_{eff} that carry variance" load-bearing. The `Mistral-7B-v0.3 / WikiText-103` perplexity table circulated in the supporting note (Vangara & Gopinath, April 2026) reports the same qualitative pattern; we register that table as a supporting-note observation, not as paper-valid evidence, because no schema-validated Mistral perplexity artifact is present under `results/v3/modal/` at the time of writing (`docs/supporting_notes/perplexity_mechanism_note_2026-04.md`, "Evidence status").

What is and isn't claimed in this subsection. The mechanism statements (M1)–(M4) are derivations from standard rate–distortion theory (Cover and Thomas, 2006; Gao and Ganguli, 2015) applied to the d_{eff} structure documented in V1-RESULT-001 and to the JL correction of Zandieh et al. (2024). The numeric attention-cosine deltas are quoted from the three-way matrix of `docs/full_matrix_evidence_summary.md` §3 and the underlying paper-valid JSON. The depth-wise compounding picture is explanatory, not directly measured; the qualitative shape of the Mistral perplexity story (data-oblivious arm collapses at low b; SPECTRALQUANT degrades gracefully) is supporting-note-only and is not used to back any numeric paper claim.

7.3 Mechanism behind the attention-output cosine matrix

The three-way attention-output cosine matrix is the most directly mechanistic measurement reported in the manuscript: it is the per-(layer, head) cosine between the FP16 attention output and each quantizer's post-cache attention output, sliced at matched compression ratio. Read literally, the cosine is the inner product of two unit-normalized hidden-state vectors at fixed layer depth, which is the same object the next layer's attention reads as input. The mechanism account therefore has three components.

(C1) Per-layer SNR is set by the post-rotation per-coordinate allocation. A data-oblivious rotation places per-coordinate variance at unpredictable locations in the rotated basis: under a Haar-random R, the marginal variance on coordinate i is $\frac{1}{d_h}\sum_j \lambda_j$ in expectation but spread widely across realizations. A uniform b-bit scalar quantizer applied on top of R pays a distortion of order $\sigma_i^2 \cdot 4^{-b}$ per coordinate; on a non-isotropic source where $\lambda_0/\lambda_{d_{\text{eff}}} \approx 10^1$ to 10^2, this means the high-variance coordinates are quantized *at the same per-coordinate precision* as the low-variance ones, and the high-variance coordinates dominate the inner product. The result is a per-(layer, head) attention-output cosine that is set by the worst-case per-coordinate distortion in the rotated basis, which is what we read as 0.40–0.66 in the TURBOQUANT columns of Table 4. The calibrated rotation puts variance into d_{eff} named coordinates and the water-filled allocation spends precision proportional to λ_i, so the per-(layer, head) inner product is preserved up to the small distortion introduced on the tail (which is at the JL noise floor by construction).

(C2) Water-filled allocation matters most at low b at fixed compression ratio. The Gaussian rate–distortion solution $D = K \cdot 2^{-2R} \cdot \prod_i \lambda_i^{1/K}$ for the d_{eff}-channel reverse-water-filling problem (Cover and Thomas, 2006, ch. 10) predicts that water-filling and uniform agree to within $O(1/B)$ in the per-coordinate distortion when $B = b \cdot d_{\text{eff}}$ is large relative to the eigenvalue spread, and binds when B is small. The matrix is the discrete realization of this prediction: $\Delta(\text{SPECTRALQUANT} - \text{uniform}) \leq +0.006$ at $b \in \{3, 5\}$ on Mistral, $+0.018$ at $b = 2$, and the ratio columns confirm the comparison is at fixed compression ($\leq 0.013\times$ apart). The mechanism is that at $b = 2$ on Mistral the uniform-allocation special case spends two bits on every semantic dimension regardless of eigenvalue, which underspends the top two coordinates and overspends the

bottom two; the water-filled rule reallocates the marginal bits toward the dominant directions, which carry most of the per-(layer, head) attention score.

(C3) What cosine can and cannot predict. The matrix is a local layer-by-layer signal; downstream metrics are functions of the full residual-stream trajectory through depth. Cosine therefore *predicts the sign* of downstream effects (preserved high-variance subspace ⇒ preserved attention output ⇒ next layer reads a cleaner residual) and the *relative ordering* of methods (SpectralQuant cosine > uniform cosine > TurboQuant cosine implies the same order on perplexity, generation F1, LongBench macro on the operating points where downstream quality has not yet collapsed). It does not predict the *magnitude* of the perplexity gap, because that magnitude depends on the 28-layer composition of cosine-deficits and on how non-linearly the next-token logits respond to residual-stream perturbation past a method-specific threshold; the TurboQuant → 2,049 collapse on Qwen2.5-7B at $b=3$ versus SpectralQuant's +7.5% over FP16 is consistent with such a threshold but is not derivable from the per-layer cosine alone. Caveat: the cosine is averaged over 8 sampled layers and 32 calibration sequences at single seed; multi-seed bands and full-depth coverage are open work.

7.4 Mechanism behind generation quality

Greedy generation reads two related but distinct signals out of the post-cache attention output: (a) the argmax of the next-token logits at each step, which determines whether the prefix the model conditions on stays on the FP16 trajectory or diverges; and (b) the shape of the next-token distribution beyond the argmax, which determines whether the generation looks well-calibrated or degenerate. Token-overlap F1 against the FP16 reference is a coarse readout of (a) and distinct-n is a readout of (b).

(G1) F1 reads attention-score SNR through the argmax. The argmax token at step t is invariant to small perturbations of the next-token logit vector as long as the gap between the top two logits exceeds the perturbation. Calibrated, water-filled quantization preserves the high-variance attention directions, so the top-1 logit is preserved at most steps; SpectralQuant's F1 of 0.482 versus the FP16 reference is consistent with a per-step argmax-flip probability of order 10^{-2} to 10^{-1}, after which the prefix diverges and the remaining $128-t$ tokens are scored against an FP16 trajectory the model is no longer on. The data-oblivious TurboQuant arm hits F1 0.120, consistent with a much higher per-step argmax-flip rate that pushes prefix divergence to the first few tokens and produces the looping pattern visible in the artifact.

(G2) Distinct-n reads distribution shape. A model whose attention output is corrupted at the high-variance directions but preserved at the low-variance ones produces softmax probabilities that re-sharpen onto a few high-frequency tokens, because the residual stream's principal contributions to the unembedding are attenuated. This is the textbook degenerate-decoding failure mode (Holtzman et al., 2020): low distinct-2, repeated n-grams, loops. SpectralQuant's distinct-2 of 0.603 (78% of FP16's 0.768) indicates a small downward shift of distribution shape, consistent with the SNR account; TurboQuant's 0.301 (39%) is the standard degenerate-decoding signature. Distinct-3 follows the same pattern in the artifact, so the effect is not a one-off bigram artefact.

(G3) Cumulative drift compounds the per-step perturbation. Greedy decoding is a deterministic function of the prefix; once the prefix diverges from the FP16 trajectory, all subsequent token distributions are conditioned on a different prefix and the comparison to the FP16 reference becomes increasingly noisy. F1 is therefore a *lower* bound on the per-step distribution fidelity — a method that preserves the per-step argmax with high probability but flips it once early in the trajectory will look worse on F1 than its actual per-step accuracy. This is one reason the generation

evidence corroborates rather than independently confirms the perplexity finding: both are downstream of the same per-(layer, head) attention-fidelity SNR, and both inherit the same single-seed / single-bit / single-model caveat. An additional caveat that is specific to generation: the eight prompts span summarization, factoid QA, and code synthesis, but a small prompt set under-samples generation tasks where attention-cosine sensitivity differs across token positions (e.g. long-form generation of a structured object), so the F1 number should be read as a directional cosine-corroboration signal rather than a generation-quality benchmark.

7.5 Mechanism behind the LongBench task split

The five-task LongBench subset splits cleanly into three retrieval-style QA tasks (`narrativeqa`, `qasper`, `hotpotqa`) where SpectralQuant matches or beats FP16 macro, and two distribution-style tasks (`gov_report`, `trec`) where SpectralQuant regresses slightly. The split is not random; it matches what the SNR / effective-subspace account predicts.

(L1) Retrieval/QA tasks live in the high-variance memory subspace. A QA task on a long context requires attention to retrieve a small number of evidence spans from thousands of tokens. Mechanistic studies of attention heads in trained transformers (Olsson et al., 2022) show that retrieval heads concentrate their effective rank on a small number of directions tied to the copy/match operation; in the calibrated eigenbasis these are exactly the high-λ coordinates that the participation-ratio gap identifies. Calibrated, water-filled quantization preserves those coordinates at high per-dim precision; the retrieval operation therefore preserves the correct evidence-span attention weights, and the QA answer follows. `qasper`'s +15.6 qa_f1 jump is the largest because its retrieval target (a short factual span inside a long scientific paper) is the cleanest case; `narrativeqa`'s +0.029 jump is smaller because narrative QA additionally requires multi-span reasoning that lives partly in the tail; `hotpotqa`'s near-tie (+0.002) is consistent with multi-hop retrieval already saturating the high-variance subspace under FP16 and not gaining additional headroom from quantization-as-regularization.

(L2) Distribution-style tasks rely on tail coordinates that quantization attenuates. `gov_report` (extractive summarization) is scored by ROUGE against a reference summary, and the choice of which sentences to include depends on discourse-level relationships that distribute across many low-variance coordinates of the key covariance — exactly the coordinates the calibrated allocation deprioritizes. A small attenuation of those coordinates produces a small (−0.022 ROUGE) but real regression. `trec` (few-shot classification) is scored by exact-match on a class label, and the model's choice of class label depends on a calibrated output-token distribution where the top few logits are close together; small attention-cosine perturbations move the argmax across a class boundary and the EM score drops in steps (−0.040). The two tasks are not failure cases of the algorithm so much as they are the operating regime where calibrated, sharp allocation is the wrong inductive bias.

(L3) Why macro > FP16 must be read cautiously. The +14.2% relative macro gain is real on this subset but not a universal LongBench claim. Three reasons. First, the gain is concentrated on `qasper`; remove that task and the macro becomes ≈ FP16 within noise. Second, *compression as regularization* is a known but speculative mechanism: a quantizer can act as an information-bottleneck regularizer that removes spurious low-variance attention paths, and on a single seed this can appear as a small QA improvement that disappears under multi-seed averaging or larger evaluation sets. Third, the deterministic five-task subset over-weights retrieval-style tasks relative to the full 21-task LongBench; tasks like `passage_count`, `passage_retrieval`, and the synthetic NIAH-style tasks have different eigenstructure dependence that a five-task subset cannot predict. The +15.6 qa_f1 jump

on `qasper` is therefore reported as the load-bearing per-task observation; the macro number is a directional signal, not a benchmark headline.

7.6 Mechanism behind the latency split

The latency artifact reports two operating points that diverge by three orders of magnitude: a K/V compress+decompress microbench at 0.06 ms/token at ctx = 1024 and a hooked-replay end-to-end at ≈ 630 ms/token at the same context. The split is not noise; it is a direct consequence of the integration boundary at which the compression kernel is invoked.

(T1) The microbench measures the kernel. The K/V compress+decompress round-trip on H200 tensors fuses (a) the calibrated rotation by $U_{\ell,h}$, (b) the per-dimension Lloyd–Max codebook lookup, (c) the codebook-back-to-FP16 inverse, and (d) the inverse rotation. All four operate on the rotated K/V tensor in HBM and have arithmetic intensity dominated by the rotation matmuls. 0.06 ms/token at ctx = 1024, scaling sublinearly in context, with zero peak-memory delta is what the mathematical work of compression actually costs once the kernel is isolated from per-layer Python control flow. The structural 5.33× memory advantage carries through the kernel intact.

(T2) Hooked replay measures Python overhead, not the kernel. The hooked-replay path wraps the same kernel in per-layer Python forward hooks that re-enter the Python interpreter, copy state in and out of the cache, and re-launch the attention kernel on a per-token basis. At ctx = 2048 this resolves to ≈2,000 Python-side callbacks per generated token, each incurring ∼0.3 ms of interpreter and CUDA-launch overhead; the resulting 630 ms/token wall clock is dominated by callback overhead, not by either the attention kernel or the compression kernel. The same shape applies to TurboQuant hooked replay (≈ 70 ms/token) modulo a constant factor: TurboQuant's simpler per-coordinate decompression is faster per callback, but the dominant term in both cases is the callback budget rather than the compression work.

(T3) What systems integration must change. The path from the microbench number to a production end-to-end speedup is not better quantization. Three changes are required and none are contributed by this manuscript. First, the per-layer Python hook must be replaced with a Hugging Face `Cache` subclass or a pre-attention rewrite that keeps control flow inside the C++/CUDA runtime. Second, decompression must be fused into the attention tile loop so that compressed K/V is decoded directly inside the kernel rather than through materialized FP16 intermediates; this is what preserves the 5.33× memory-bandwidth advantage past the attention prologue. Third, the compression metadata (codebooks, rotation matrices, per-(layer, head) d_{eff}) must be resident in the SM register file or shared memory at attention time to avoid re-fetching it. Until these land, the latency evidence supports only the kernel-level claim. We therefore explicitly do not report a production-kernel end-to-end latency win in this manuscript; the structural 5.33× memory factor is the load-bearing serving-side claim, and an end-to-end speedup is open work.

7.7 Mechanism behind the compression accounting

Matched compression ratio is what makes the rest of the paper a causal-attribution argument. The accounting block in `src/spectralquant/accounting.py` enforces this at the schema level by recording original payload bytes, compressed payload bytes, codebook bytes, JL projection storage, calibration metadata, and per-(layer, head) d_{eff} on every artifact.

(A1) Why matched ratio matters for causal attribution. If SpectralQuant spent more bits per element than TurboQuant or than the uniform-allocation special case, the quality delta

could be explained by the budget rather than by the algorithm. The three-way matrix's "Unif. ratio" and "SpectralQuant ratio" columns are within 0.013× on every operating point, and the four next-stage runs all stamp ≈3 bits/element on SpectralQuant and TurboQuant alike. The schema-level constraint is therefore that the accounting field must agree across methods on the same row of the same artifact; any post-hoc claim of a "win" that does not pass this constraint would be a budget claim, not an algorithm claim.

(A2) Why the simple $16/b$ story is not the right accounting. The naive compression ratio for b-bit scalar quantization of FP16 keys is $16/b$ (5.33× at $b = 3$). The headline 5.95× on Qwen2.5-14B-Instruct is larger, and the spec §10 formula does not yield 5.95× from $b=3$ and $d_{\text{eff}}=3$ alone. Three additional terms drive the gap. First, the per-(layer, head) d_{eff} varies; some heads have $d_{\text{eff}}=2$ and contribute a smaller per-element semantic payload, while the tail at the JL noise floor is dropped or quantized below b. Second, the per-dimension Lloyd–Max codebooks are stored once per (layer, head, bit-budget) and amortize across all calibration tokens; their per-vector cost is small but nonzero and must be charged in the denominator. Third, the JL projection storage and rotation-matrix metadata are constant per (layer, head) and amortize over the cache size, but they are not free for short contexts. The accounting module computes the actual ratio across all four terms; the headline number is therefore the schema-validated total, not the back-of-the-envelope $16/b$.

(A3) What this changes for downstream comparisons. A future end-to-end claim — production-kernel latency, multi-seed quality bands, full LongBench — must run through the accounting module rather than through a back-of-the-envelope budget. Two implications. First, the matched-ratio constraint is a hard gate for any future cross-method comparison: a method that beats SpectralQuant at $b = 2$ but spends 1.5× more bytes per element does not beat SpectralQuant at the matched compression ratio, and the accounting module is what catches that. Second, the codebook/rotation amortization changes the operating regime: at very short context ($L \lesssim d_{\text{eff}}$) the per-vector overhead dominates and compression buys less than $16/b$; at long context the asymptotic ratio is approached. Long-context inference is the regime where the KV cache is the binding constraint, so the long-context asymptote is the relevant operating point, but a serving system that multiplexes short and long contexts must include the amortization cost in its planner.

7.8 Mechanism behind the participation-ratio universality

The first-round headline is that $d_{\text{eff}}/d_h \approx 3$–$5\%$ on every Hugging Face transformer we calibrated (Qwen2.5-1.5B/7B/14B, Mistral-7B-v0.3, Llama-3-8B). This is the empirical hook on which the rest of the paper rests, and a mechanism-level reading is in order.

(D1) Why d_{eff} is small. The participation ratio is sensitive to the eigenvalue spread: equal eigenvalues give $d_{\text{eff}} = d_h$, single-direction concentration gives $d_{\text{eff}} = 1$. Trained attention heads concentrate their per-(layer, head) key covariance into a small number of high-variance directions, because the attention operation itself is rank-selective: the softmax amplifies whichever inner products are largest, which biases the gradient to push key updates onto a small number of axes. Mechanistic studies of induction heads, name-mover heads, and other function-specific heads (Olsson et al., 2022) are consistent with this picture: the functional rank of a head is much smaller than d_h. The 3–5% band is therefore a quantitative restatement of a known qualitative phenomenon.

(D2) Why a small calibration set suffices. The calibration is ≈15 seconds on a single GPU because the per-(layer, head) covariance needs to estimate only a $d_h \times d_h = 128 \times 128$ matrix, and the spectrum collapses past $d_{\text{eff}} \approx 4$ so the top-d_{eff} eigenvectors are stable under modest sample sizes. The calibration objects in the repository are computed from 32–256 sequences depending on artifact,

well above the rank-d_{eff} identifiability threshold. Where calibration may fail is on out-of-distribution prompt distributions whose eigenstructure differs from the calibration distribution; we have not measured this, and a domain-shift calibration audit is open work.

(D3) Caveats on universality. The five-model universality is empirical, not theoretical. We do not have a proof that d_{eff}/d_h is bounded for all transformers; the 3–5% band could break on a model with explicit isotropy-promoting regularization, on a much larger model where the per-head dimension grows with depth, or on a model trained on a distribution with different rank structure (e.g. multimodal inputs). Multi-architecture coverage is open work; the existing universality result is the necessary, not sufficient, evidence for the calibrated-rotation approach.

8 Limitations and remaining work

We name every limitation rather than soften it. Each item below is a gate in `docs/evidence_catalog.md`; resolution requires a named artifact landing schema-valid on disk and clearing the `paper_valid` gates.

1. **Multi-seed.** Every SpectralQuant number reported in Sections 6.2–6.6 is single seed 42. Multi-seed runs on at least perplexity and three-way attention cosine are the highest-priority next step (catalog gap `V1-GAP-001`; the identifier is historical, the gap is current).

2. **Production-kernel latency.** The hooked-replay end-to-end rows are slower than FP16. To make a production speedup claim, the SpectralQuant engine needs a Hugging Face `Cache` subclass or a pre-attention rewrite that avoids per-layer Python callbacks [evidence: `docs/claims_discipline.md` §5.2.4; runbook §7.7.5].

3. **Official / optimized TurboQuant comparator.** Every TurboQuant arm in this manuscript is our implementation of the published TurboQuant method (`src/spectralquant/spectralquant.py::TurboQuantBaseline`). The official Google reference (Zandieh et al., 2025) and the Blackwell-cuTile port (Vangara, 2026) are not vendored or run here; consequently the latency rows compare two unoptimized PyTorch baselines, and a production-default claim requires running against the optimized reference. V1-GAP-012 and V1-GAP-013 remain in force.

4. **Selective-QJL ablation matrix.** The three-way attention-cosine matrix isolates two contributions (calibrated rotation vs. random rotation, and water-filled vs. uniform allocation *within* the calibrated rotation), but does not separate the contribution of *selective* QJL from the rest of the pipeline. The five-cell ablation that would close this gap is (a) eigenbasis + uniform allocation + full QJL, (b) eigenbasis + water-filled allocation + full QJL, (c) eigenbasis + water-filled allocation + *no* QJL, (d) random rotation + same per-dimension Lloyd–Max codebooks (no selective-QJL split, isolating the rotation contribution under matched per-dim quantization), and (e) raw (un-rotated) basis + water-filled allocation. Until those runs land, the per-component attribution between the calibrated rotation, the water-filled allocation, the per-dimension Lloyd–Max codebooks, and the selective-QJL split remains blurry; the manuscript's mechanistic claim is the joint contribution of these ingredients at matched compression ratio.

5. **Value-side calibration audit.** The present implementation reuses the K-side eigenbasis $U_{\ell,h}$ and the same d_{eff} split for V and quantizes V with per-dimension Lloyd–Max codebooks at the same bit budget. Whether a separately calibrated V-eigenbasis (or a per-(layer, head) mixed allocation rule across K and V) buys additional attention-output cosine is not measured in this manuscript; the value-side question therefore enters the headline numbers as a fixed engineering choice rather than as a controlled ablation.

6. **Distributional law for keys.** The Gaussian rate–distortion model of equation (3) is a design heuristic, not an empirical claim about LLM key marginals. We have not measured Gaussianity of the per-coordinate marginals after the calibrated rotation, and the robustness of the water-filled allocation rule to non-Gaussian heavy-tailed marginals is not isolated; Lloyd–Max handles the empirical scalar marginal in any case but the closed-form rate–distortion derivation that motivates water-filling assumes a near-Gaussian source. A direct distributional audit (Gaussianity tests, Lloyd–Max-vs-uniform-codebook robustness on the empirical marginals) is open work.

7. **LongBench full path.** The deterministic five-task subset is paper-valid; the headline must say "5-task LongBench subset". Full 21-task LongBench remains unblocked.

8. **Multi-architecture under the next-stage harness.** Sections 6.2–6.5 are single-model (Qwen2.5-7B). Mistral-7B-v0.3 runs are scheduled. The Mistral-WikiText-103 perplexity table that circulates in the supporting note `docs/supporting_notes/perplexity_mechanism_note_2026-04.md` is registered as supporting-note observation only and is not used to back any numeric claim in this manuscript; promotion to paper-valid requires a schema-validated artifact under `results/v3/modal/perplexity__Mistral-7B-v0.3_*.json`.

9. **Calibration cost amortization.** Calibration takes wall-clock seconds per layer × head; observability events record this but the amortization-curve evidence is not yet a paper-valid artifact (V1-GAP-007).

10. **Two engines in namespace.** Catalog gap `V1-GAP-010`: `src/spectralquant/engine.py` and `src/spectralquant/spectralquant.py` coexist. The expanded-layer technical spec uses only the canonical one; the deprecated engine should be removed before publication.

11. **Pre-existing test failures.** Six calibration/quantization test failures predate the consolidation work, are unrelated to the four next-stage families, and are tracked in the `V1-GAP-*`/`V2-GAP-*` table in `docs/evidence_catalog.md`. They do not affect any number reported here.

9 Discussion and forward-looking implications

The body of evidence above is a quality-validity argument for one post-training KV-cache quantizer at a small set of operating points. The implications, if the SNR / effective-subspace account holds up under multi-seed and multi-architecture scrutiny, run further than that. This section sketches what those implications would be. We mark every claim in this section explicitly: *evidence-backed* where it is restating something already shown above, *mathematically derived* where it follows from rate–distortion or linear-algebra arguments applied to the empirical d_{eff} structure, and *speculative / future work* where it is a forward-looking implication that this manuscript does not validate. The section is intended as a research-direction map, not as a list of claims about the present.

9.1 Distribution-aware inference

The first implication, and the one closest to the present evidence, is that a small, cheap calibration pass on a representative input distribution is enough to discover the per-(layer, head) high-variance subspace that attention actually uses. The cost is measured in seconds (initial evidence layer; the amortization curve is open work), and the benefit is a per-(layer, head) eigenbasis into which a quantizer, an attention sparsifier, an eviction policy, or a selective-precision kernel can project before deciding where to spend bandwidth. This is *evidence-backed* in the narrow sense that the calibrated rotation already buys +0.27 to +0.38 attention cosine over a data-oblivious one at matched compression ratio (Section 6.6); it is *mathematically derived* that the eigenbasis is the variance-maximizing rotation under a Gaussian rate–distortion problem (Section 2.4); it is *speculative*

that the same calibration object can be reused across non-quantization compressors (e.g. low-rank cache projections, H2O-style (Zhang et al., 2023) eviction policies, or attention- sparsity kernels) without re-deriving each. We expect the speculation to hold because all of those compressors operate on the same K covariance, but no integrated system has been built and measured against this manuscript's harness yet.

A practical near-term version is a Hugging Face `Cache` subclass that takes a calibration tensor, computes per-(layer, head) $U_{\ell,h}$ and $d_{\text{eff},\ell,h}$, and stores compressed keys in that basis at request-time. The serving cost is dominated by the calibration pass, which is amortized over the serving window; incoming requests at long context inherit the variance-aware allocation without paying calibration latency per request. The production-kernel speedup discussed in Section 6.5 is the precondition: until decompression is fused into the attention tile loop, the memory-bandwidth advantage of the calibrated compression does not reach end-to-end serving.

9.2 Training-time implications

If LLM keys are strongly non-isotropic at convergence, that is a property of the model that future training procedures could *target* rather than passively observe. Several speculative directions follow.

SNR-aware regularization. A training objective could include an auxiliary penalty on the participation ratio of each attention head's empirical key covariance over a short rolling window, encouraging the model to concentrate variance into a small number of coordinates by construction. The motivation is that a model that converges to a smaller d_{eff} is intrinsically more compressible at inference time, and existing post-training quantizers like SpectralQuant would inherit better operating points without any additional inference-time work. This is *speculative*: we have not run such a training experiment, and it is plausible that penalizing d_{eff} during training degrades model quality by removing representational capacity that the model uses for tail behaviors. A careful version of this experiment would be a small-scale ablation on a 1B-parameter model with a controlled d_{eff}-penalty and a matched-compute baseline.

Explicit allocation of representational bandwidth. The water-filling rule of equation (3) treats the bit budget as a scarce resource to be allocated across coordinates; in a training context, the analogous scarce resource is gradient signal per coordinate. Architectures could allocate channel-wise width or attention-head dimensionality non-uniformly across layers based on a calibrated estimate of where information gets stored, mirroring the inference-time calibrated rotation. Mixture-of-experts work (Fedus et al., 2022) already routes capacity dynamically; the speculation here is that a similar discipline applied to attention head dimensionality based on per-head d_{eff} targets could buy a parameter-count efficiency the way SpectralQuant buys a bit-budget efficiency. *Speculative; future work.*

Compression-as-regularization. The single-seed LongBench result that SpectralQuant macro-beats FP16 by +14.2% relative on the deterministic five-task subset (Section 6.4) is consistent with a story in which mild compression noise on low-variance coordinates acts as an inductive bias that helps QA tasks while mildly hurting summarization and classification. If that mechanism is real, it raises the possibility that compression-style regularizers applied during training could buy similar quality gains without needing to be applied at inference time. This direction overlaps with classical work on noise injection during training and with dropout (Srivastava et al., 2014); the contribution of the present work, if any, would be a principled, calibrated, per-(layer, head) allocation rather than a uniform isotropic one. *Speculative; treat the LongBench macro-beats-FP16 number as a directional signal, not as a license for a regularization claim.*

9.3 Domain-specific compression and effective subspaces

The participation-ratio universality result of the initial evidence layer (NVIDIA Research, 2025) was measured on Qwen2.5-1.5B/7B/14B, Mistral-7B-v0.3, and Llama-3.1-8B—natural-language transformers trained on broadly similar text corpora. Two extensions are *speculative* but worth measuring:

Biological-sequence transformers. Protein language models such as ESM-2 (Lin et al., 2023) and the genomic models in the AlphaMissense (Cheng et al., 2023) and Evo (Nguyen et al., 2024) families are trained on amino-acid or nucleotide sequences with very different distributional structure from natural-language text. The d_{eff}/d_h ratio in these models is an empirical question we have not measured here; if it is small, the same SpectralQuant pipeline would compress those caches at the same ratio, and the calibration object becomes a measurement tool for the domain itself: which residue positions does the model actually attend to, and along which directions. If the ratio is *not* small, that is itself an interesting finding—it would mean that biological-sequence transformers use a wider effective key subspace than natural-language ones, and that long-context biological inference would not benefit from the same compression. We register this as future work; the supporting note observes the hypothesis but no schema-validated artifact exists for biological models.

Clinical and long-context legal/scientific corpora. Domains with long, structured documents (clinical notes, legal contracts, scientific papers) are exactly the cases where KV-cache compression is most operationally valuable, because their contexts routinely exceed 32,768 tokens. Whether the calibrated d_{eff} structure learned on a generic web-crawl corpus transfers to these domains, or whether per-domain calibration improves the per-(layer, head) eigenbasis, is a measurable question. The calibration cost is small enough that per-domain calibration is feasible without retraining; the open question is the quality delta it buys. *Future work.*

Calibration as a measurement tool. The most general form of this implication is that the per-(layer, head) eigenbasis $U_{\ell,h}$ and the spectrum $\lambda_{\ell,h,1} \geq \ldots \geq \lambda_{\ell,h,d_h}$ are not just compression preprocessing; they are a low-cost, model- and data-specific summary of what each attention head has learned to attend to. A calibration pass on a domain-specific corpus produces a measurement of how that corpus is represented in the model's attention machinery, independent of any compression decision. We expect this view to be useful for interpretability and for diagnosing distribution shift between training and inference; *speculative*, but the calibration object already exists and is cheap to produce.

9.4 Interpretability: spectra as a diagnostic

The calibrated eigenbasis at each (layer, head) yields a small number of principal directions in the $d_h = 128$-dimensional key space. Three uses of this object are *mathematically derived* or *evidence-backed*: (i) the participation ratio d_{eff} is a scalar summary of how spread the head's key distribution is across coordinates, (ii) the eigenvalues $\lambda_{\ell,h,i}$ rank coordinates by how much the head varies along them, and (iii) the eigenvectors $u_{\ell,h,i}$ are explicit directions in the key space. Two further uses are *speculative* but follow naturally:

- **Mechanistic-interpretability probes.** The dominant eigenvectors of the K covariance can be projected onto the embedding space (or a probe trained on top of it) to ask what linguistic or semantic property the head's high-variance direction encodes. This complements existing probing work (Tenney et al., 2019; Elhage et al., 2022) by anchoring the probe to the head's empirical variance structure rather than to a chosen task-specific feature. *Future work; we have not run such probes here.*

- **Cross-layer compositionality.** The set of eigenvectors across layers $\{U_{\ell,h}\}$ is a low-rank description of what the residual stream stores at each depth. If the calibrated eigenbases align across layers in a structured way, that is a measurement of how the model passes information through depth; if they rotate freely, that is a different statement about representation geometry. The calibration objects to support this analysis already exist in the repository for the five tested transformers; the analysis is *future work*.

9.5 Transformer architecture: why d_{eff} is small

A second-order question the evidence raises but does not answer: *why* is d_{eff}/d_h in the 3–5% band, and almost the same band, across very different transformer families? Two non-exclusive hypotheses, both *speculative*:

Softmax attention as an information bottleneck. The softmax over inner products $q^\top k_i$ concentrates probability mass on a small number of keys; for the gradient to flow through that bottleneck, the keys must be distinguishable along a small number of directions in inner-product space. A small effective key subspace is the optimization-friendly solution. If this is correct, then d_{eff} is set by the structure of softmax attention itself and we should expect any architecture that retains softmax-over- inner-products to inherit a similar d_{eff}/d_h ratio. Architectures that replace softmax with linear attention (Katharopoulos et al., 2020) or with state-space mixers (Gu and Dao, 2023) would test this hypothesis directly: do their analogous covariance objects have similar participation ratios?

Grouped-query attention as a forcing function. Models with $n_{\text{kv}} \ll n_{\text{q}}$ via GQA (Ainslie et al., 2023) share each key head across multiple query heads. The shared-head constraint is plausibly a forcing function pushing keys onto a narrower subspace, since the same key tensor must be useful across several attention computations. The participation-ratio measurements in this work are taken on GQA models; non-GQA models or multi-query attention (MQA) (Shazeer, 2019) would be a controlled test of this hypothesis. *Future work.*

These are research questions, not claims. The point of registering them here is that the SpectralQuant pipeline produces, as a side effect of calibration, a quantitative summary object (d_{eff}, Λ, U per layer-head) that is well-suited to studying them.

9.6 Systems roadmap

The shortest path from this manuscript to a serving-time win is *not* another quality experiment; it is the systems work already named in Section 8. We list the sequence here once, in priority order, with the SNR / effective- subspace framing that organizes them:

1. **Production cache subclass.** A Hugging Face `Cache` subclass or pre-attention rewrite that removes the per-layer Python forward hooks; this lets the K/V microbench 0.06 ms/token kernel reach end-to-end serving without the hooked-replay overhead.
2. **Fused decompression.** Decompression fused into the attention tile loop so that compressed KV is consumed directly by the attention kernel without a materialized FP16 intermediate; this is the change that preserves the 5.33× memory-bandwidth advantage end-to-end. Blackwell-cuTile (Vangara, 2026) is one reference point.
3. **Multi-seed and multi-architecture.** Multi-seed runs on at least perplexity and three-way attention cosine, plus the expanded-layer harness extended to Mistral-7B-v0.3, Llama-3-8B, and at least one larger model. This is the gate that converts the single-seed directional LongBench signal into either a real macro-beats-FP16 result or a corrected smaller delta.

4. **Full 21-task LongBench.** The deterministic five-task subset is paper-valid as a subset claim; the full suite is the only way to validate the per-task interpretation in Section 6.4 (QA tasks helped, summarization / classification mildly hurt) at the population level.

5. **Official Google / Blackwell-cuTile TurboQuant comparator.** Our implementation of TurboQuant is sufficient to demonstrate the calibrated-vs-data-oblivious gap as a research finding; an optimized reference baseline (the official Google (Zandieh et al., 2025) or the Blackwell-cuTile port (Vangara, 2026)) is required before any production-default claim.

6. **Low-level kernels.** Beyond the cache subclass, a fused per-tile water-filled-decode kernel on Hopper / Blackwell is a longer-horizon target. The mathematical compression has been demonstrated; the systems compression is what converts it into serving cost.

The single sentence summary: the present manuscript establishes the quality validity of calibrated, water-filled compression at matched ratio; the next year of work converts that validity into production performance and broader empirical coverage.

10 Reproducibility

10.1 Repository layout

- Repository: `niashwin/spectralquant-full` (private), `main`.
- Specification: `docs/spectralquant_v2_technical_spec.md`.
- Manuscript: `paper_output_consolidated/spectralquant_unrestricted_paper.tex` and the compiled `paper_output_consolidated/spectralquant_unrestricted_paper.pdf`.
- Bibliography: `paper_output/spectralquant_refs.bib`.
- Modal runbook: `docs/execution_audit_and_modal_runbook.md` §7.7.

10.2 Result JSONs and schemas

Every empirical sentence in this manuscript points to one of the following artifacts. All five validate against the schemas in `schemas/` and carry `paper_valid = true`.

- Perplexity: `results/v3/modal/perplexity__Qwen2.5-7B__b3_seed42_n1024_tok1024_str512_fp16+spectralquant_v2+turboquant.json`.
- Generation: `results/v3/modal/generation__Qwen2.5-7B__b3_seed42_t0.00_new128_fp16+spectralquant_v2+turboquant.json`.
- Latency: `results/v3/modal/latency__Qwen2.5-7B__b3_seed42_bs1_ctx512x1024x2048_gen64_wm3_it10_fp16+spectralquant_v2+turboquant.json`.
- LongBench: `results/v3/modal/longbench__Qwen2.5-7B__b3_seed42_subsetdeterministic_n50_in8192_out128_fp16+spectralquant_v2+turboquant.json`.
- Three-way attention cosine: artifacts under the Modal volume `spectralquant-v2-results:/results/three_way/`, summarized verbatim in `docs/full_matrix_evidence_summary.md` §3.

10.3 Modal launch commands

Each run's exact command is committed verbatim into the artifact JSON `command` field; extract with `jq .command results/v3/modal/<file>.json`. The headline commands are:

```
# Perplexity
modal run -d scripts/launch_modal_eval.py::run_perplexity \
  --model Qwen/Qwen2.5-7B --b3 --seed 42 \
  --n-eval-sequences 1024 --max-eval-tokens 1024 --stride 512

# Generation
modal run -d scripts/launch_modal_eval.py::run_generation \
  --model Qwen/Qwen2.5-7B --b3 --seed 42 \
  --temperature 0.0 --max-new-tokens 128

# Latency
modal run -d scripts/launch_modal_eval.py::run_latency \
  --model Qwen/Qwen2.5-7B --b3 --seed 42 \
  --batch-size 1 --gen-tokens 64 \
  --context-lengths 512,1024,2048 \
  --include-microbench --include-end-to-end-replay \
  --warmup-iters 3 --measure-iters 10

# LongBench (deterministic 5-task subset)
modal run -d scripts/launch_modal_eval.py::run_longbench \
  --model Qwen/Qwen2.5-7B --b3 --seed 42 \
  --subset deterministic --n-per-task 50 \
  --max-input-tokens 8192 --max-new-tokens 128
```

10.4 Schema validation

Validation is exercised by `tests/test_eval_paper_valid_gates.py` (29 assertions) and `tests/test_longbench_partial_persist.py` (5 assertions). Every artifact must pass: `mode = full` (real HF weights, real corpus); for non-FP16 methods, `replay_coverage.fraction_layers_real` ≥ 0.99; family-specific token counts (perplexity $\geq$ 64,000 tokens; LongBench $n_{\text{per\_task}} \geq 50$; latency $n_{\text{iters}} \geq 10$ measured); `device = cuda`; CUDA-event timer for latency. `paper_valid = true` is only written if every gate holds; `false` carries an explicit `caveats[]` array.

10.5 Provenance and security

Artifacts contain no Hugging Face tokens, Modal tokens, or other secrets; they record `software.{torch, transformers, datasets, python}` versions, GPU info, and model class only. Reproduction needs the operator's HF token (Mistral-7B-v0.3 is gated) and Modal CLI tokens, managed per `docs/modal_safety_protocol.md` §3—never via CLI flags, never echoed into the result JSON, and never committed.

10.6 Figure regeneration

Every figure in this manuscript is produced by `scripts/build_paper_figures.py`, which reads the JSON artifacts above and the verbatim three-way table in `docs/full_matrix_evidence_summary.md` §3 and writes `paper_output_v2/figures/*.pdf`. The script is deterministic under matplotlib 3.10 and produces six figures (`fig_perplexity.pdf`, `fig_generation.pdf`, `fig_longbench.pdf`, `fig_latency.pdf`, `fig_attention_cosine.pdf`, `fig_pipeline.pdf`). Empirical numbers in figures are *not* hand-typed; they are loaded from the JSONs at build time.

11 Claim-to-artifact traceability

Every empirical sentence in this manuscript carries an explicit `[evidence: ...]` pointer; Table 5 groups them by section. Sentences not in this table are methodological narrative or derivations.

Table 5: Claim-to-artifact map. Sources are docs/evidence_catalog.md and the JSONs in results/v3/modal/.

Sec.	Claim (one-line)	Evidence ID / repo path
§1, §2.3	$d_{\text{eff}}/d_h \approx 3{-}5\%$ on tested models	V1-RESULT-001, V1-IMPL-001 @ results/memory_efficiency/all_models.json, src/spectralquant/calibration.py
§6.6	Three-way attention-cosine table	RUN-THREEWAY-MISTRAL-{2,3,5}BIT, RUN-THREEWAY-QWEN-3BIT @ docs/full_matrix_evidence_summary.md §3
§6.1, §6.2	Perplexity (6.4907/6.9773/2,048.5671)	RUN-PERPLEXITY-QWEN2.5-7B @ results/v3/modal/perplexity__Qwen2.5-7B__b3_seed42_*.json
§6.1, §6.3	Token-F1 0.482/0.120; distinct-2 0.603/0.301	RUN-GENERATION-QWEN2.5-7B @ results/v3/modal/generation__Qwen2.5-7B__b3_seed42_*.json
§6.1, §6.4	LongBench macro 0.20041/0.17547/0.00442	RUN-LONGBENCH-QWEN2.5-7B-DETERMINISTIC @ results/v3/modal/longbench__Qwen2.5-7B__b3_seed42_*.json
§6.1, §6.5	Latency rows + `production_kernel = false`	RUN-LATENCY-QWEN2.5-7B @ results/v3/modal/latency__Qwen2.5-7B__b3_seed42_*.json
§4.3 (compression accounting)	Uniform/water-filled ratios within 0.013×	docs/full_matrix_evidence_summary.md §3, src/spectralquant/accounting.py
§4.4	Replay coverage 1.0 (28/28)	RUN-PERPLEXITY-QWEN2.5-7B methods.spectralquant_v2.replay_coverage
§4.5	Calibration mode + observability	commit 98e559f; experiments/sqv2_replay.py, experiments/run_longbench.py, runbook §7.7.4a
§6.4	Per-method partial persistence + recovery merger	commit 1ecb578; experiments/run_longbench.py _write_method_partial_record, scripts/merge_longbench_partials.py, tests/test_longbench_partial_persist.py
§6.7	Compression-accounting derivation	V1-IMPL-007, V1-GAP-014 @ src/spectralquant/accounting.py, docs/evidence_catalog.md
§8	Open-gap catalog	V1-GAP-* @ docs/evidence_catalog.md
§7.2	Mechanism narrative (M1–M4); Mistral perplexity qualitative shape	Math derivations (Section 2.4, eq. 3) and supporting note docs/supporting_notes/perplexity_mechanism_note_2026-04.md (Sentra technical note, Vangara & Gopinath, April 2026); the note's Mistral-WikiText-103 perplexity table is registered as supporting-note-only, not paper-valid

12 Conclusion

Long-context inference is bound by the KV cache, and the dominant academic recipe for compressing it—a data-oblivious random rotation followed by uniform-bit scalar quantization—pays a quality penalty that is not just an accident of any particular implementation but a consequence of the strong non-isotropy of LLM key vectors. The manuscript supports four core claims at clearly bounded scope: (i) the per-(layer, head) KV-key covariance is low-effective-rank ($d_{\text{eff}}/d_h \approx 3$–$5\%$) on every Hugging Face transformer we calibrated; (ii) SpectralQuant replaces the random rotation with the calibrated eigenbasis and the uniform allocation with greedy water-filling, at the same compression ratio, sharing the K eigenbasis with the matched V tensor; (iii) at matched compression ratio, SpectralQuant strictly beats our implementation of TurboQuant on attention-output cosine (+0.27 to +0.38 on the four three-way operating points), and the water-filled allocation strictly improves the uniform special case at $b=2$ on Mistral-7B-v0.3 (+0.018 mean cosine), where the rate–distortion calculus predicts the allocation rule should bind; (iv) Qwen2.5-7B at $b=3$ holds perplexity, generation token-F1, and a deterministic five-task LongBench subset macro at FP16-class levels, with the LongBench macro-beats-FP16 number reported as a directional signal driven primarily by `qasper` rather than as a universal LongBench claim. Production-kernel end-to-end latency (which requires fusing decompression into the attention tile loop, not better quantization), multi-seed confidence intervals, full 21-task LongBench, an official Google (Zandieh et al., 2025) or Blackwell-cuTile (Vangara, 2026) comparator, multi-architecture coverage on the four next-stage families, the selective-QJL ablation matrix, and a value-side calibration audit are explicit open work. The repository `niashwin/spectralquant-full` is sufficient to reproduce every empirical sentence in this manuscript.

Conflict of interest

The authors are affiliated with Sentra (commercial inference) and MIT. This work does not depend on any Sentra-internal codebase; the artifacts in the repository `niashwin/spectralquant-full` are sufficient to reproduce every claim in this manuscript.

Acknowledgments

We thank the maintainers of the Hugging Face Transformers ecosystem, the Modal team for the H200 serving environment, and the authors of the rotation-based KV-quantization line of work for clean public implementations against which SpectralQuant can be compared.

References

Dimitris Achlioptas. Database-friendly random projections: Johnson-Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003. URL https://doi.org/10.1016/S0022-0000(03)00025-4.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL https://arxiv.org/abs/2305.13245.

Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2404.00456.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding, 2023. URL https://arxiv.org/abs/2308.14508.

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, et al. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2024. URL https://arxiv.org/abs/2412.15204.

Sambartha Ray Barman, Andrey Starenky, Sofia Bodnar, Nikhil Narasimhan, and Ashwin Gopinath. The price of meaning: Why every semantic memory system forgets. *arXiv preprint arXiv:2603.27116*, 2026a. URL https://arxiv.org/abs/2603.27116.

Sambartha Ray Barman, Andrey Starenky, Sofia Bodnar, Nikhil Narasimhan, and Ashwin Gopinath. The price of meaning: Why every semantic memory system forgets, 2026b.

Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-bit quantization of large language models with guarantees, 2024. URL https://arxiv.org/abs/2307.13304.

Yichi Chen et al. KVTC: KV cache compression with token-channel joint indexing. In *International Conference on Learning Representations (ICLR)*, 2026.

Jun Cheng, Guido Novati, Joshua Pan, Clare Bycroft, Akvilė Žemgulytė, Taylor Applebaum, Alexander Pritzel, Lai Hong Wong, Michal Zielinski, Tobias Sargeant, Rosalia G. Schneider, Andrew W. Senior, John Jumper, Demis Hassabis, Pushmeet Kohli, and Žiga Avsec. Accurate proteome-wide missense variant effect prediction with AlphaMissense. *Science*, 381(6664):eadg7492, 2023. doi: 10.1126/science.adg7492.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006. ISBN 978-0-471-24195-9. Chapter 10 (Rate Distortion Theory) covers the classical water-filling allocation across parallel Gaussian channels.

Haojie Duanmu, Zhihang Yuan, Xiuhong Li, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. SKVQ: Sliding-window key and value cache quantization for large language models, 2024. URL https://arxiv.org/abs/2405.06219.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. Anthropic Transformer Circuits Thread, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.

William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Hao Feng, Zhonghao Huang, Qi Zhang, Zuchao Li, Yanghua Zhang, Ruizheng Li, Xiang Ao, and Ling Yu. GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM, 2024. URL https://arxiv.org/abs/2403.05527.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.17323.

Ying Gao and Surya Ganguli. On the properties of neural machine translation: Encoder-decoder approaches. Workshop proceedings, 2015. Ganguli lab work on linear dimensionality of neural representations.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs, 2024. URL https://arxiv.org/abs/2310.01801.

Gemma Team. Gemma 2: Improving open language models at a practical size, 2024. URL https://arxiv.org/abs/2408.00118.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL https://arxiv.org/abs/2312.00752.

Ahmed Burak Gulhan, Krishna Teja Chitty-Venkata, Murali Emani, Mahmut Kandemir, and Venkatram Vishwanath. BaKlaVa: Budgeted allocation of KV cache for long-context inference, 2025. URL https://arxiv.org/abs/2502.13176.

Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV caches with polar transformation, 2025. URL https://arxiv.org/abs/2502.02617. Venue listed variously as NeurIPS 2025 / AISTATS 2026; verify before camera-ready.

Mohsen Hariri, Lam Nguyen, Sixu Chen, Shaochen Zhong, Qifan Wang, Xia Hu, Xiaotian Han, and Vipin Chaudhary. More for keys, less for values: Adaptive KV cache quantization, 2025. URL https://arxiv.org/abs/2502.15075.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1904.09751.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization, 2024. URL https://arxiv.org/abs/2401.18079.

Wei Huang, Haotong Qin, Yangdong Liu, Yawei Li, Xianglong Liu, Luca Benini, Michele Magno, and Xiaojuan Qi. SliM-LLM: Salience-driven mixed-precision quantization for large language models, 2024. URL https://arxiv.org/abs/2405.14917.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b, 2023. URL https://arxiv.org/abs/2310.06825.

William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz maps into Banach spaces, 1984. Conference on Modern Analysis and Probability.

Greg Kamradt. Needle in a haystack — pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023. GitHub repository.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 5156–5165, 2020.

Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. A survey on large language model acceleration based on KV cache management, 2024. URL https://arxiv.org/abs/2412.19442.

Xing Li, Zeyu Xing, Yiming Li, Lin Qu, Hui-Ling Zhen, Wulong Liu, Yiwu Yao, S. J. Pan, and Mingxuan Yuan. KVTuner: Sensitivity-aware layer-wise mixed precision KV cache quantization, 2025. URL https://arxiv.org/abs/2502.04420.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL https://arxiv.org/abs/2306.00978. Best Paper Award.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache, 2024. URL https://arxiv.org/abs/2402.02750.

Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2): 129–137, 1982.

Denis Malinovskii et al. HIGGS: Pushing the limits of large language model quantization via the linearity theorem, 2024. URL https://arxiv.org/abs/2411.17525. NAACL 2025.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL https://arxiv.org/abs/1609.07843.

Eric Nguyen, Michael Poli, Matthew G. Durrant, Brian Kang, Dhruva Katrekar, David B. Li, Liam J. Bartie, Armin W. Thomas, Samuel H. King, Garyk Brixi, Jeremy Sullivan, Madelena Y. Ng, Ashley Lewis, Aaron Lou, Stefano Ermon, Stephen A. Baccus, Tina Hernandez-Boussard, Christopher Ré, Patrick D. Hsu, and Brian L. Hie. Sequence modeling and design from molecular to genome scale with Evo. *Science*, 386(6723): eado9336, 2024. doi: 10.1126/science.ado9336.

NVIDIA Research. KVTC: KV cache compression via channel-wise decorrelation, 2025. ICLR 2026.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023. URL https://arxiv.org/abs/2211.05102.

Qwen Team. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

scos-lab. turboquant: Python reference implementation of TurboQuant. https://github.com/scos-lab/turboquant, 2025. Community Python reference implementation.

Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019. URL https://arxiv.org/abs/1911.02150.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56): 1929–1958, 2014.

Carsen Stringer, Marius Pachitariu, Nicholas Steinmetz, Matteo Carandini, and Kenneth D. Harris. High-dimensional geometry of population responses in visual cortex. *Nature*, 571:361–365, 2019. URL https://doi.org/10.1038/s41586-019-1346-5.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2024. URL https://arxiv.org/abs/2104.09864.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4593–4601, 2019. URL https://aclanthology.org/P19-1452.

Mart van Baalen, Christos Louizos, Markus Nagel, Rana Ali Amjad, Ying Wang, Tijmen Blankevoort, and Max Welling. Bayesian bits: Unifying quantization and pruning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2005.07093.

Anirudh Bharadwaj Vangara. turboquant_cutile: cuTile-based Blackwell implementation of TurboQuant. https://github.com/DevTechJr/turboquant_cutile, 2026. Community implementation used as baseline in this work.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.03762.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.10438.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.02694.

Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. LLM inference unveiled: Survey and roofline model insights, 2024. URL https://arxiv.org/abs/2402.16363.

Amir Zandieh, Majid Daliri, and Insu Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead, 2024. URL https://arxiv.org/abs/2406.03482.

Amir Zandieh, Majid Daliri, Ali Hadian, and Vahab Mirrokni. TurboQuant: Near-optimal data-oblivious quantization for KV cache compression, 2025. URL https://arxiv.org/abs/2504.19874. ICLR 2026.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.14048.